

VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

FOR

<https://juice-shop.herokuapp.com/#/>

FROM

SAUDA MOMIN

CONTENTS

Sr.no	Content	Page number
1.	Executive summary Summary Objective Report	4
	Tabular summary	5-7
3	Graphical Summary	8
4	Technical report	
4.1	Authentication Bypass via SQL Injection in login email field	9-12
4.2	Account Takeover of emma@juice-sh.op via Weak Security Question	13-18
4.3	Business Logic Vulnerability - Negative Quantity Order Exploit	19-23
4.4	Broken Access Control - Change Password Without Knowing Current Password	24-27
4.5	DOM XSS in Product Search	28-30
4.6	Unauthorized Access to Administrative Panel	31-32
4.7	Insecure Direct Object Reference	33-35
4.8	Cookies Missing HTTP Only Flags	36-37
4.9	XML External Entity (XXE)	38-40
4.10	File Path Traversal with Information Disclosure via Improper Error Handling	41-43
4.11	Business Logic Vulnerability – Bypass of Deluxe Membership Payment	44-47
4.12	Insecure Transmission of Credentials via HTTP	48-49
4.13	Outdated JavaScript Library – jQuery v2.2.4	50-52
4.14	Broken Access Control: Bid Value Manipulation in Auction Feature	53-55
4.15	Reflected Cross-Site Scripting (XSS) in Order Tracking Input	56-58
4.16	Business Logic Error - Multiple Discount Request via Chat Box	59-61

4.17	Null Byte Injection Allows Unauthorized Download of Backup Files	62-64
4.18	Broken Access Control – Unauthorized Deletion of 5-Star Reviews	65-67
4.19	Verbose Error Message Revealing Internal Implementation Details	68-69
4.20	Unauthenticated Access to Application Metrics Endpoint	70-72
4.21	Insecure Autocomplete configuration on Login Form	73-74
5	Conclusion	75

SUMMARY

1.1 EXECUTIVE SUMMARY

INSTITUTE OF INFORMATION SECURITY (IIS) has assigned the task of vulnerability assessment and penetration testing (VAPT) on <https://juice-shop.herokuapp.com/#/>

This VAPT was performed on 8 August 2025. The detailed report and our findings are described below.

1.2 OBJECTIVE

The objective of this report is to identify, analyze, and document the security vulnerabilities present in the OWASP Juice Shop application. This assessment aims to demonstrate common web application security flaws based on the OWASP Top 10 framework, understand their potential impact, and recommend effective mitigation strategies. The ultimate goal is to enhance awareness of web security risks and improve secure development practices by using OWASP Juice Shop as a practical learning tool.

1.3 SCOPE

The scope of our work was limited to only <https://juice-shop.herokuapp.com/#/>. The vulnerabilities and our findings are described below.

TABULAR SUMMARY

The following tables summarize the vulnerability assessment of the server.

CATEGORY	DESCRIPTION
No of live host	1
No of vulnerabilities	21
No of critical vulnerabilities	0
No of high vulnerabilities	15
No of medium vulnerabilities	1
No of low vulnerabilities	5

Fig 2.1

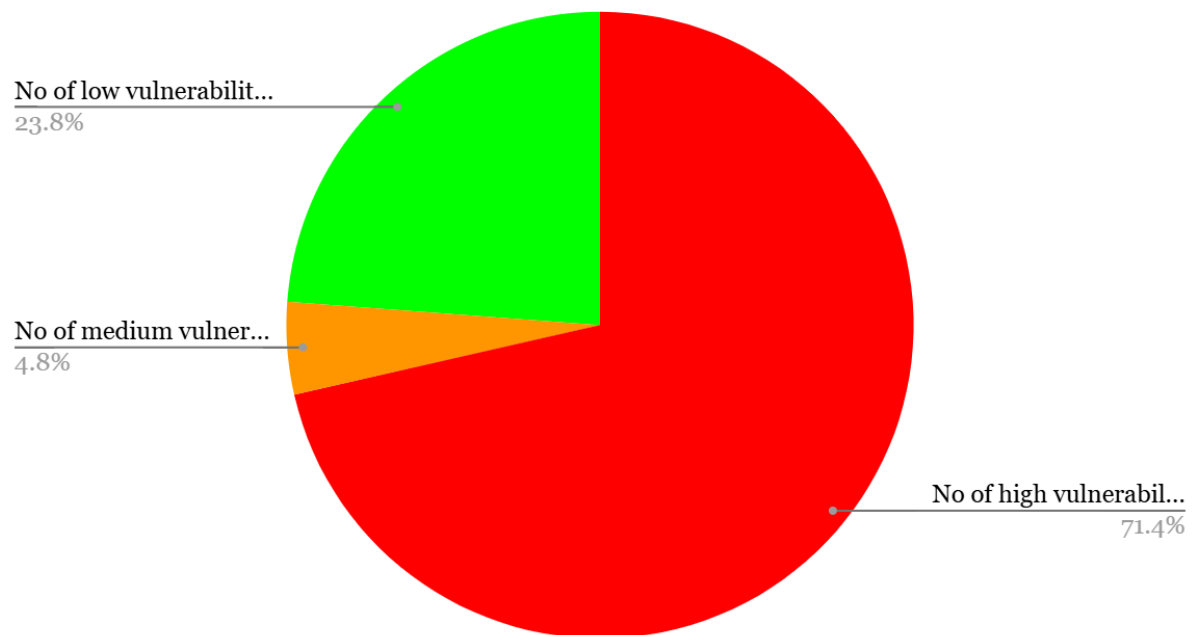
SrNo.	Vulnerability Name	Severity	Ease of Exploitation
1	Authentication Bypass via SQL Injection in login email field	High	Easy
2	Account Takeover of emma@juice-sh.op via Weak Security Question	High	Moderate
3	Business Logic Vulnerability - Negative Quantity Order Exploit	High	Easy
4	Broken Access Control - Change Password Without Knowing Current Password	High	Very easy
5	DOM XSS in Product Search	High	Medium
6	Unauthorized Access to Administrative Panel	High	Easy
7	Insecure Direct Object Reference	High	Easy
8	Cookies Missing HTTP Only Flags	High	Easy
9	XML External Entity (XXE)	High	High
10	File Path Traversal with Information Disclosure via Improper Error Handling	High	Easy
11	Business Logic Vulnerability – Bypass of Deluxe Membership Payment	High	Easy
12	Insecure Transmission of Credentials via HTTP	High	Easy
13	Outdated JavaScript Library – jQuery v2.2.4	High	Moderate
14	Broken Access Control: Bid Value Manipulation in Auction Feature	High	Moderate
15	Reflected Cross-Site Scripting (XSS) in Order Tracking Input	High	Easy

16	Business Logic Error - Multiple Discount Request via Chat Box	Medium	Easy
17	Null Byte Injection Allows Unauthorized Download of Backup Files	Medium	Easy
18	Broken Access Control – Unauthorized Deletion of 5-Star Reviews	Medium	Easy
19	Verbose Error Message Revealing Internal Implementation Details	Medium	Easy
20	Unauthenticated Access to Application Metrics Endpoint	Medium	Easy
21	Insecure Autocomplete configuration on Login Form	Low	Very easy

GRAPHICAL SUMMARY

The following pie chart graphically summaries the vulnerability assessment

Vulnerabilities by Severity Level (Percentage)



3.3 – Pie chart showing the percentage distribution of vulnerabilities by severity

1. Authentication Bypass via SQL Injection in Login Email Field

a) Classic SQL Injection Bypass

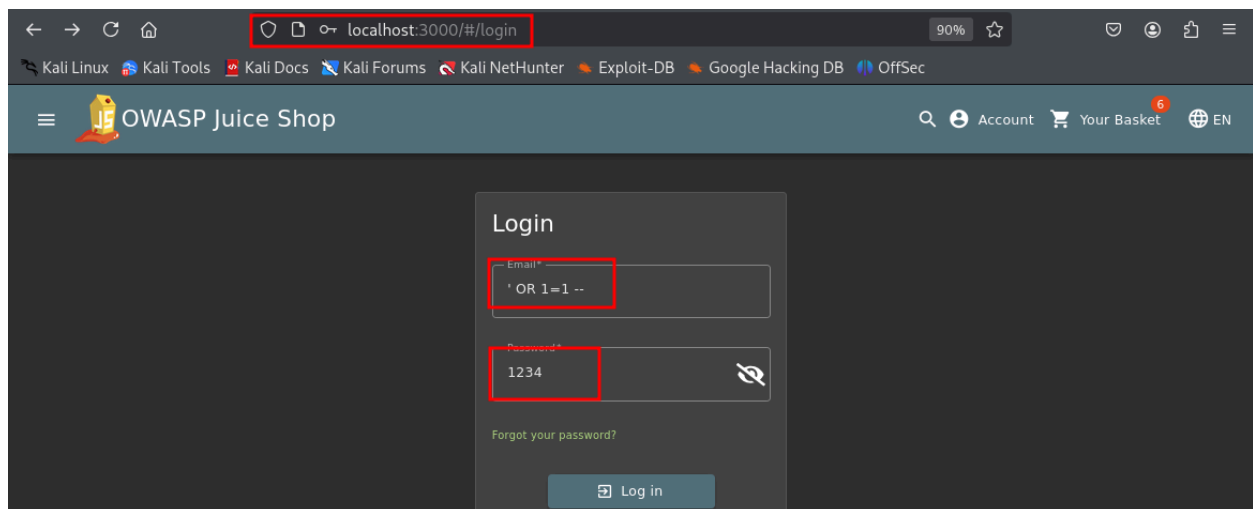
Affect on

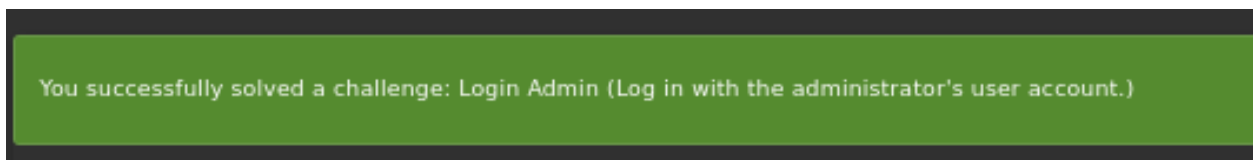
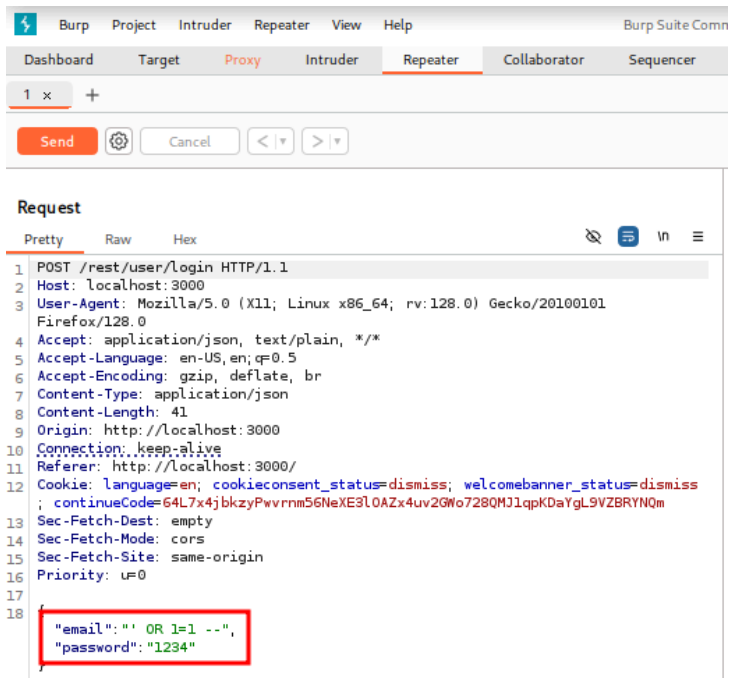
Infected URL	http://localhost:3000/#/login
Infected Parameter	Email
Parameter type	Post
Attack vector	"email": " ' OR 1=1 --", "password": "1234"

- Severity:- High
- Ease of Exploitation: Easy
This vulnerability can be exploited by anyone with basic knowledge of SQL injection, using only the login form.

Analysis

During testing, I targeted the login functionality of OWASP Juice Shop. I supplied the following payload in the email field:





with any arbitrary password (in this case, 1234). Upon submitting the request, I was successfully logged into the application, which clearly indicates that the login mechanism is vulnerable to SQL Injection.

This suggests that the server-side logic is directly embedding user input into a SQL query without using parameterized statements or input sanitization. The injected payload manipulated the SQL statement to always evaluate to true, bypassing credential checks.

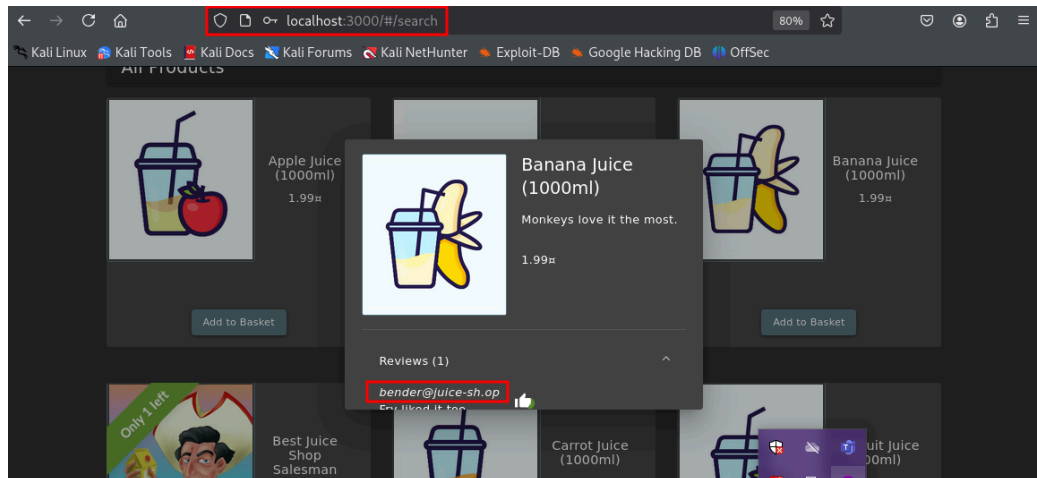
b) Targeted User Login Bypass

Affect on

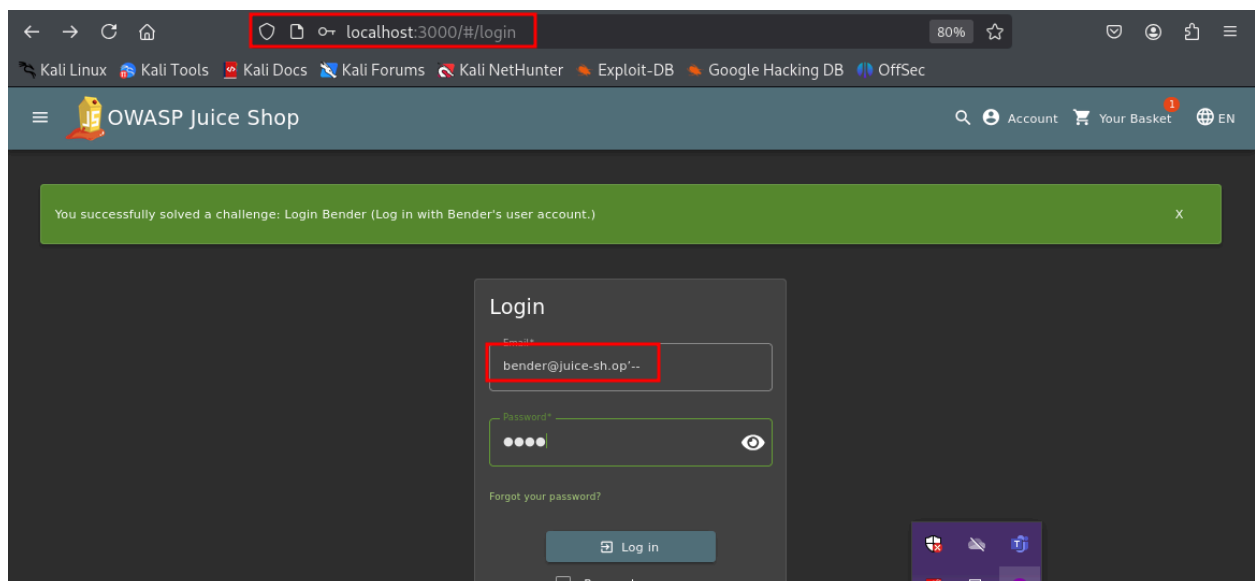
Infected URL	http://localhost:3000/#/login
Infected Parameter	Email
Parameter type	Post
Attack vector	"email": " <u>bender@juice-sh.op</u> '-- "password": "1234"

Analysis

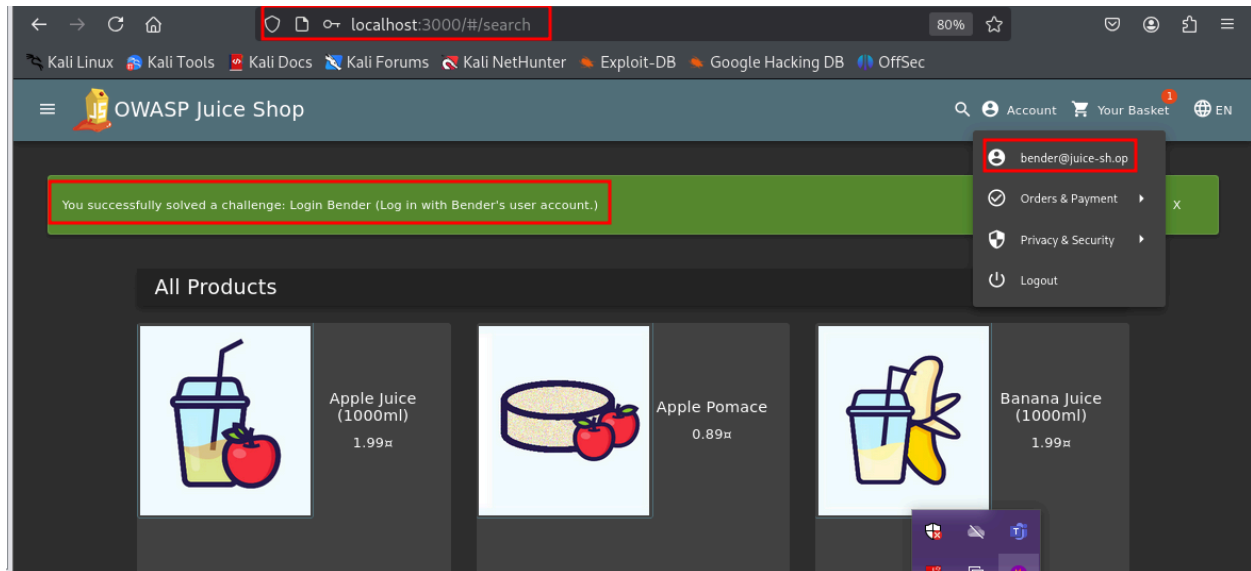
While performing the penetration test on OWASP Juice Shop, I inspected the application's publicly available data and discovered the email address **bender@juice-sh.op** exposed in the customer reviews section.



Using this email, I crafted a SQL Injection payload in the login form's email input field by appending a SQL comment sequence: `bender@juice-sh.op'--`



The “--” comment operator effectively ignored the password condition, bypassing password validation.



As a result, I successfully logged into the account of the user **bender@juice-sh.op** without knowing the actual password.

Root Cause:

- Absence of parameterized queries
- Lack of input validation on user-supplied data

Impact:

- Unauthorized login as any user (including administrative accounts)
- Potential for full application compromise
- Possible exposure of sensitive data
- Risk of privilege escalation, data manipulation, or additional SQL injection exploits

Mitigation:

- Use parameterized queries (prepared statements) in all database interactions
- Avoid direct string concatenation when building SQL queries
- Implement server-side input validation and sanitization
- Employ Object-Relational Mapping (ORM) libraries that enforce safe query practices
- Use Web Application Firewalls (WAFs) to help detect and block SQL injection attempts

2. Account Takeover of emma@juice-sh.op via Weak Security Question

Affect on

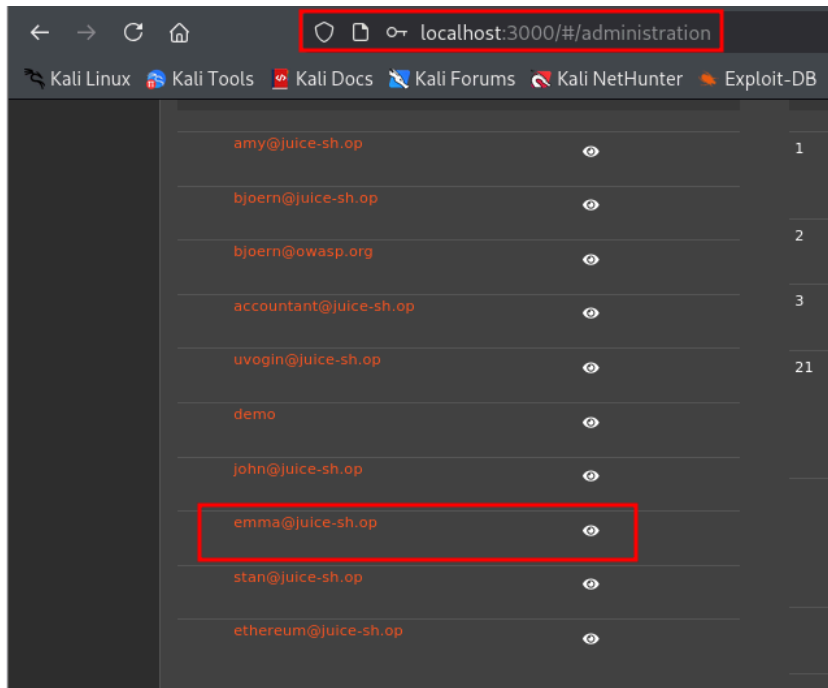
Infected URL	POST /rest/user/reset-password
Infected Parameter	• email — identifies the user account to reset • securityAnswer — the answer to the security question used for verification • newPassword — the new password to set
Parameter type	Post
Attack vector	An attacker uses the password reset feature by entering the victim's email. When prompted for the security question, the attacker guesses the answer based on clues found in the victim's publicly uploaded image. Providing the correct answer allows the attacker to reset the password and gain unauthorized access to the account.

- Severity:- High: Allows account takeover without knowing original password, compromising user confidentiality and integrity.
- Ease of Exploitation :- Moderate

Analysis

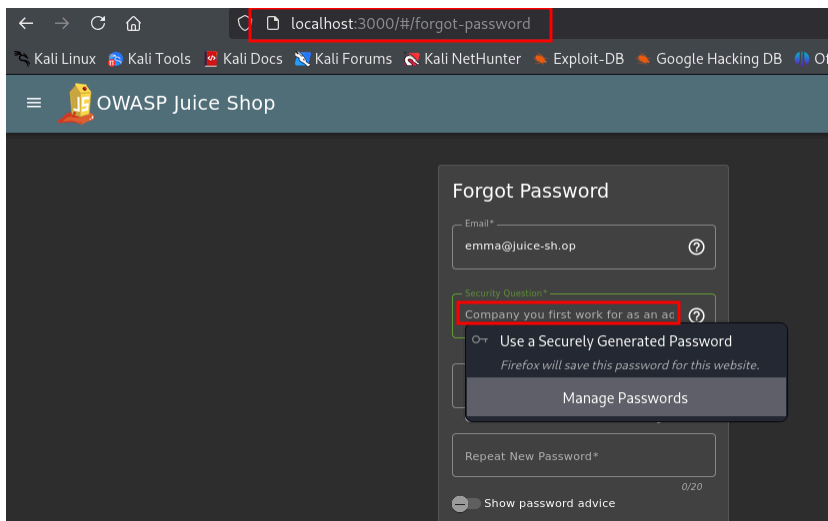
During testing, I identified an account takeover vulnerability affecting the user emma@juice-sh.op via the password reset functionality.

Initially, I obtained Emma's email address by accessing the **administration account**, which has privileges to view user information including registered emails. This allowed me to confirm the exact email tied to Emma's account (emma@juice-sh.op).



When attempting to reset Emma’s password using the **“Forgot Password”** feature, the application prompted me to answer the security question:

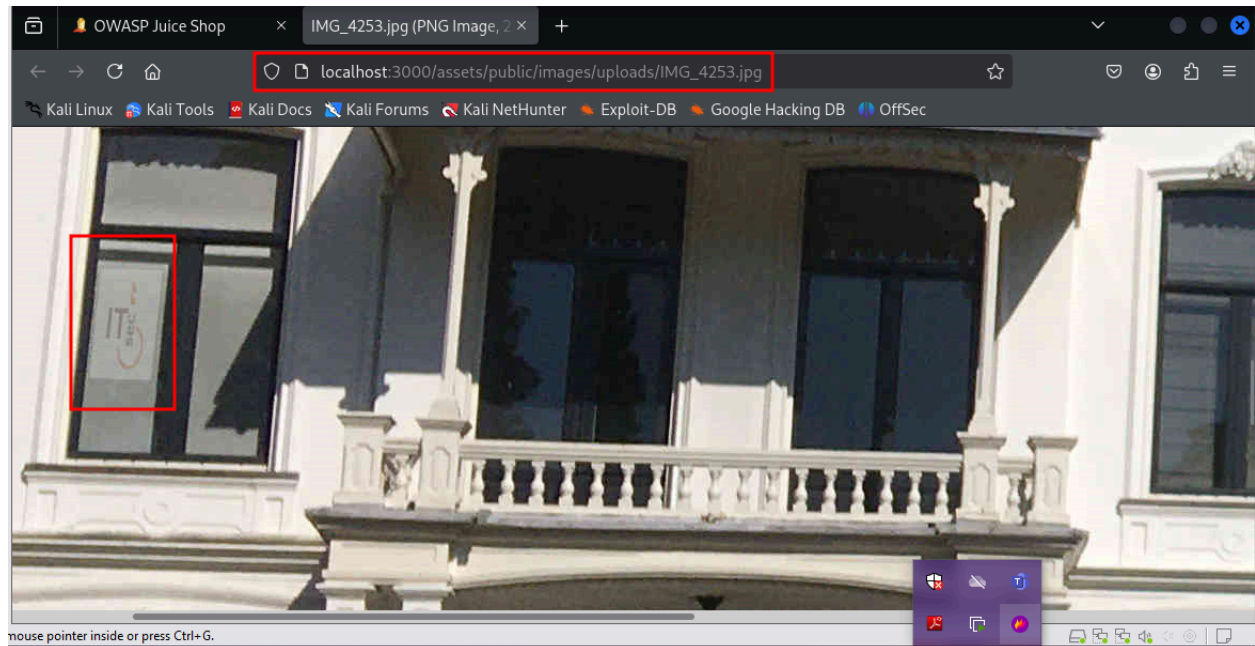
“Company you first work for as an adult?”



Since I did not know the answer, I investigated Emma's publicly available content to find clues. Emma had uploaded an image captioned **"my old work place"**.

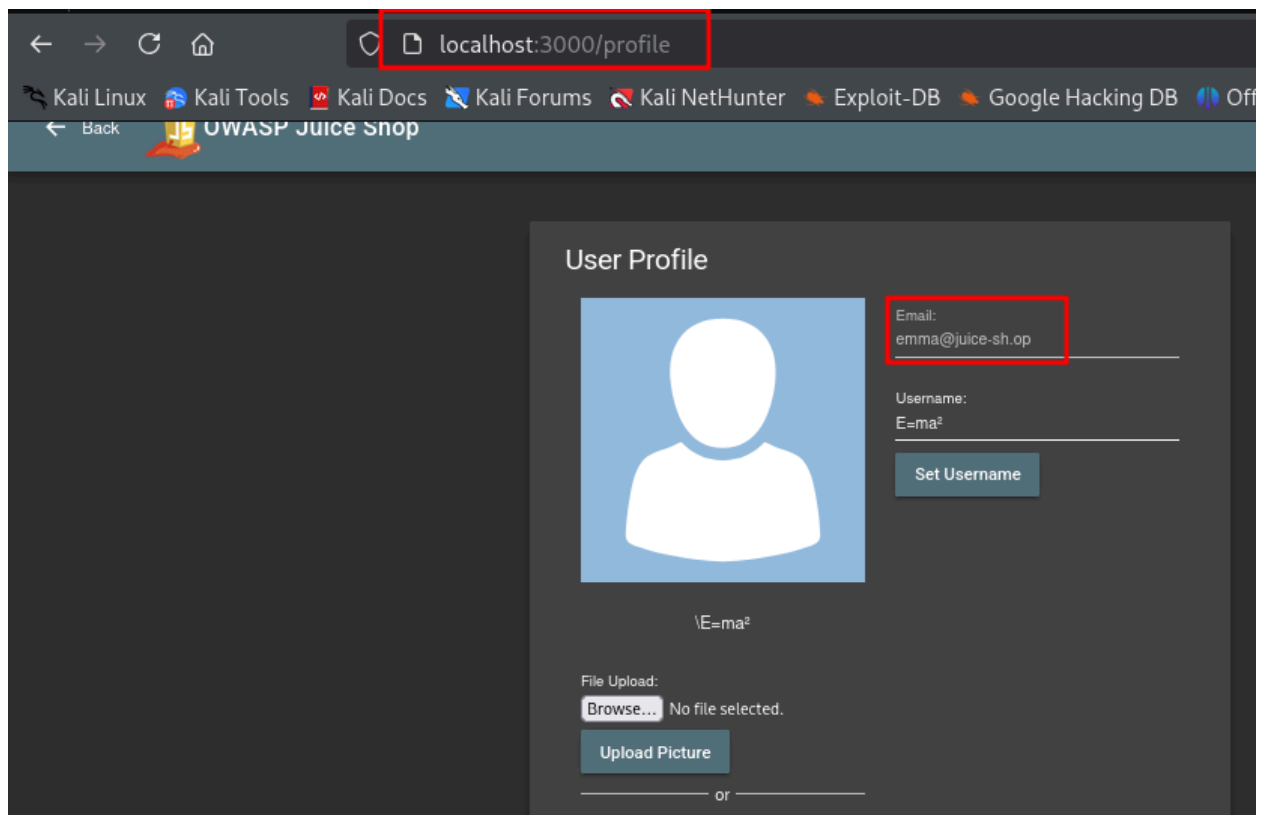
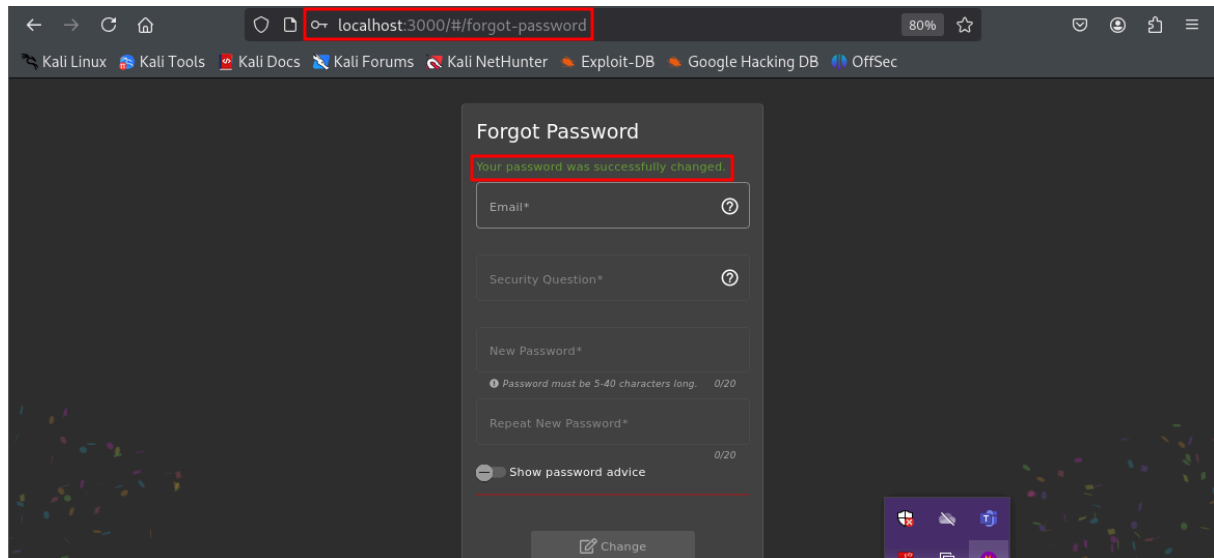


By zooming in on this image, I discovered text in a window that read something like **"IT sec"**, which appeared to be a clue to the answer for the password reset security question: **"Company you first work for as an adult?"**



I then attempted several variations of this phrase as answers, including:

- IT Sec
- ITSec
- IT sec
- ITsec



You successfully solved a challenge: Visual Geo Stalking (Determine the answer to Emma's security question by looking at an upload of her to the Photo Wall and use it to reset her password via the Forgot Password mechanism.)

The correct answer was **ITsec**, which enabled me to reset the password and gain access to Emma's account without knowing the original password.

Root Cause

- Using easily guessable or publicly exposed answers for security questions.
- Insufficient protections on password reset endpoints.

Impact

- Unauthorized access to Emma's account.
- Potential access to sensitive user data or privileged functions.
- Violates user privacy and data security principles.

Mitigation

- Replace security questions with stronger multi-factor authentication (MFA).
- Avoid using user-supplied content for security question answers.
- Implement rate limiting and account lockout policies on password reset attempts.
- Educate users not to upload sensitive clues about their security questions.
- Consider removing security questions altogether in favor of email-based password reset links.

3. Business Logic Vulnerability - Negative Quantity Order Exploit

Affect on

Infected URL	http://localhost:3000/api/BasketItems/{id}
Infected Parameter	quantity
Parameter type	Put
Attack vector	Intercept the PUT /api/BasketItems/{id} request and modify the JSON "quantity" parameter to a negative value (e.g., -100), causing the application to apply a credit instead of charging the user.

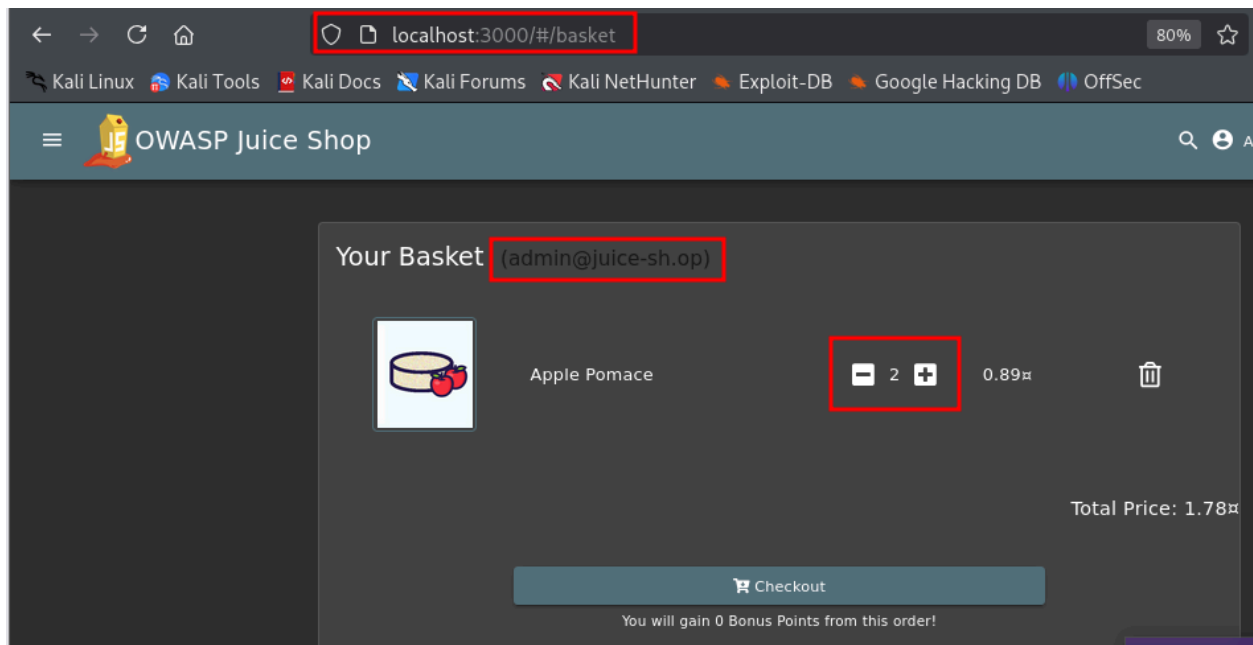
- Severity:- High – Leads to direct financial loss and exploitation without privileged access.
- Ease of Exploitation:- Easy – Requires only a standard account and basic interception tools (e.g., Burp Suite, browser DevTools).

Analysis

During testing of the OWASP Juice Shop application, it was discovered that the basket update functionality fails to validate quantity inputs on the server side.

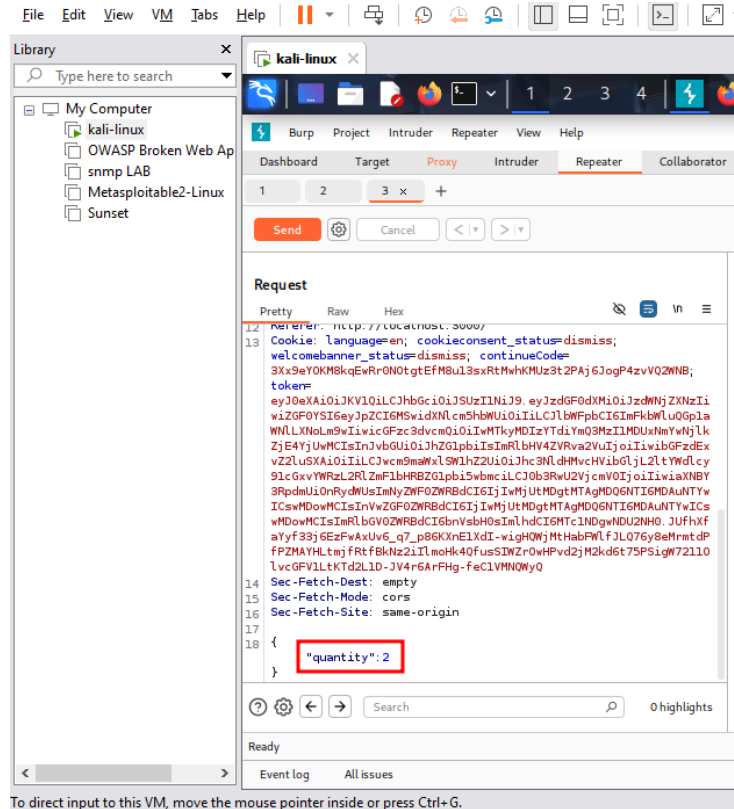
By directly manipulating API requests, a logged-in user can set a **negative quantity** for an item, causing the total order value to become negative and crediting the attacker instead of charging them.

While testing the shopping basket functionality, I logged in as a regular user and added an item to my basket.

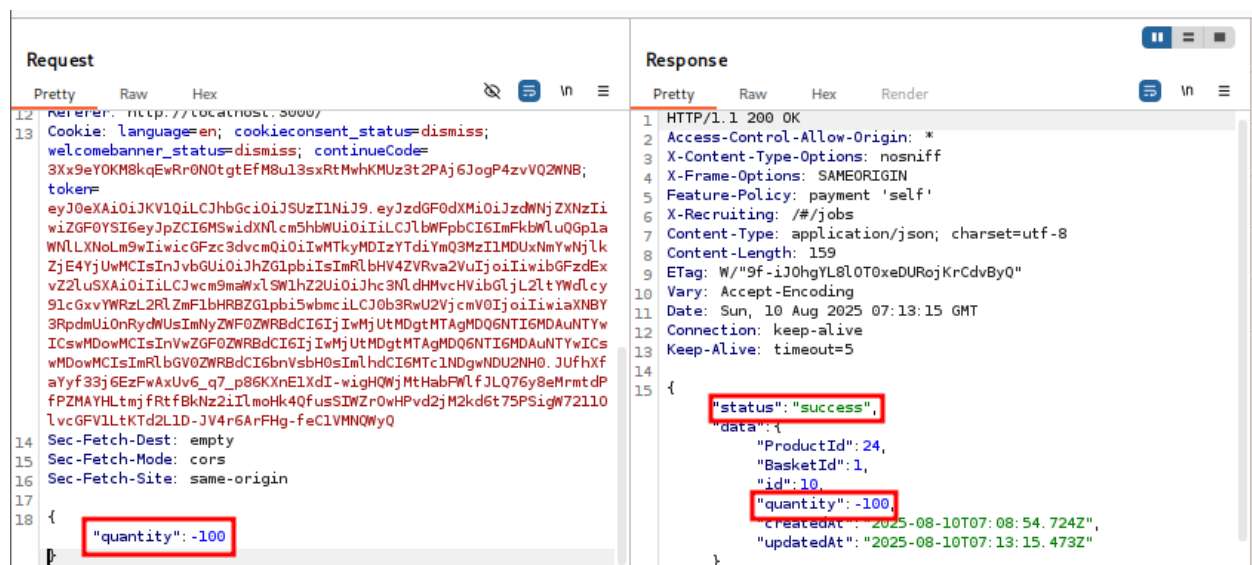


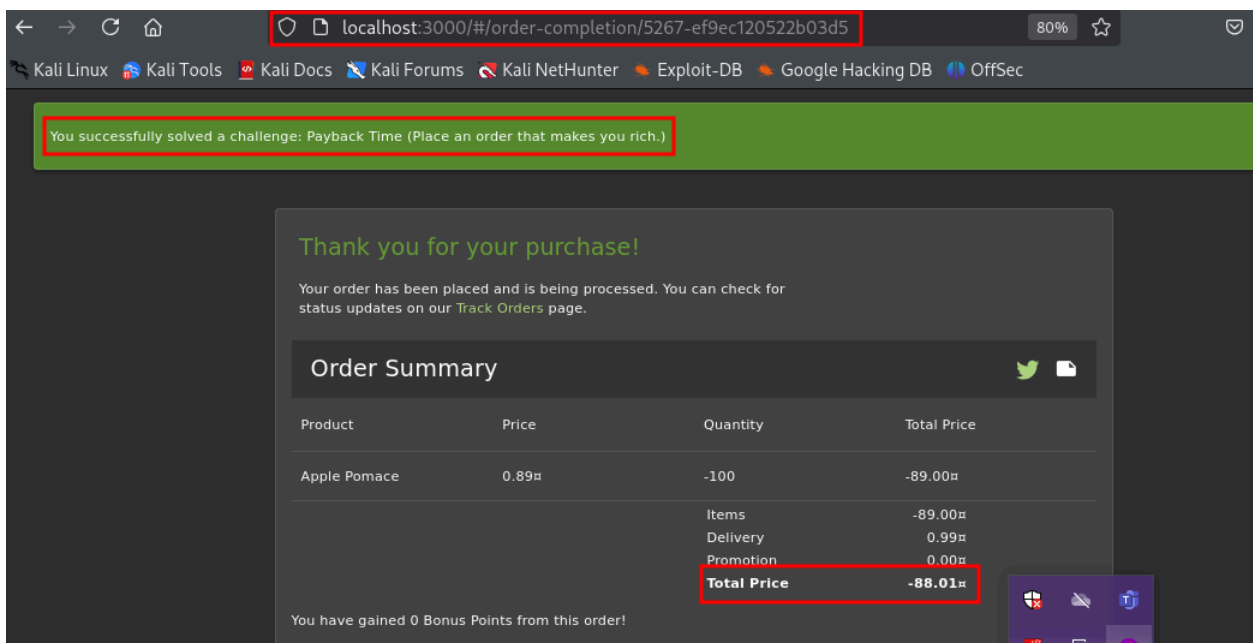
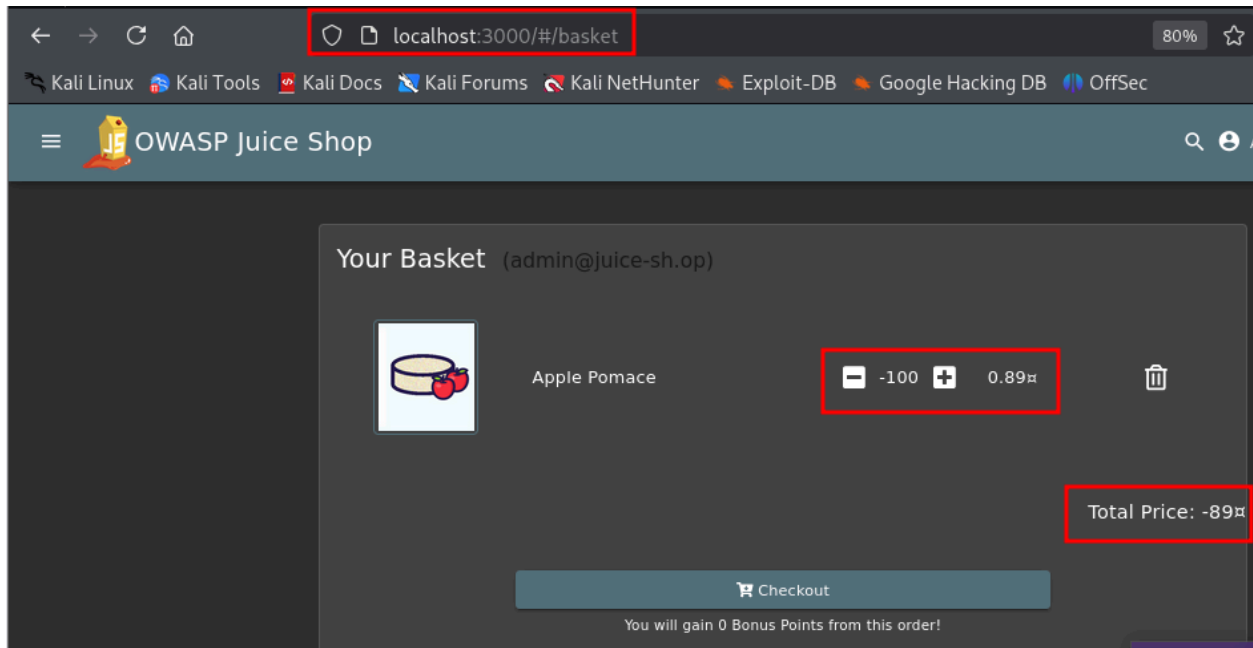
I then intercepted the traffic using **Burp Suite**. When I changed the quantity of the item in the UI, I captured a PUT request to: `/api/BasketItems/{id}`

The request body contained: `{"quantity": 2}`



I modified the "quantity" value to -100 directly in Burp Suite and forwarded the request to the server. The application accepted this negative quantity without any validation, which caused my basket total to become negative and credited my account with the difference.





This confirmed that there was no **server-side validation** of the quantity parameter, and the client-side restriction preventing values below 1 could be bypassed entirely.

Root Cause:

- Lack of server-side input validation for quantity values.
- Reliance on client-side controls that can be bypassed.

Impact:

- Attackers can generate unlimited store credit.
- Potential financial loss to the business.
- Abuse of inventory and accounting systems.
- Erodes trust in the platform.

Mitigations:

1. Implement strict server-side validation to ensure quantity > 0 .
2. Reject requests with invalid quantity values and return an error.
3. Add business logic checks in the backend to prevent negative pricing scenarios.
4. Implement server-side logging and monitoring for abnormal order values.
5. Use automated security tests to detect similar business logic flaws.

4. Broken Access Control - Change Password Without Knowing Current Password

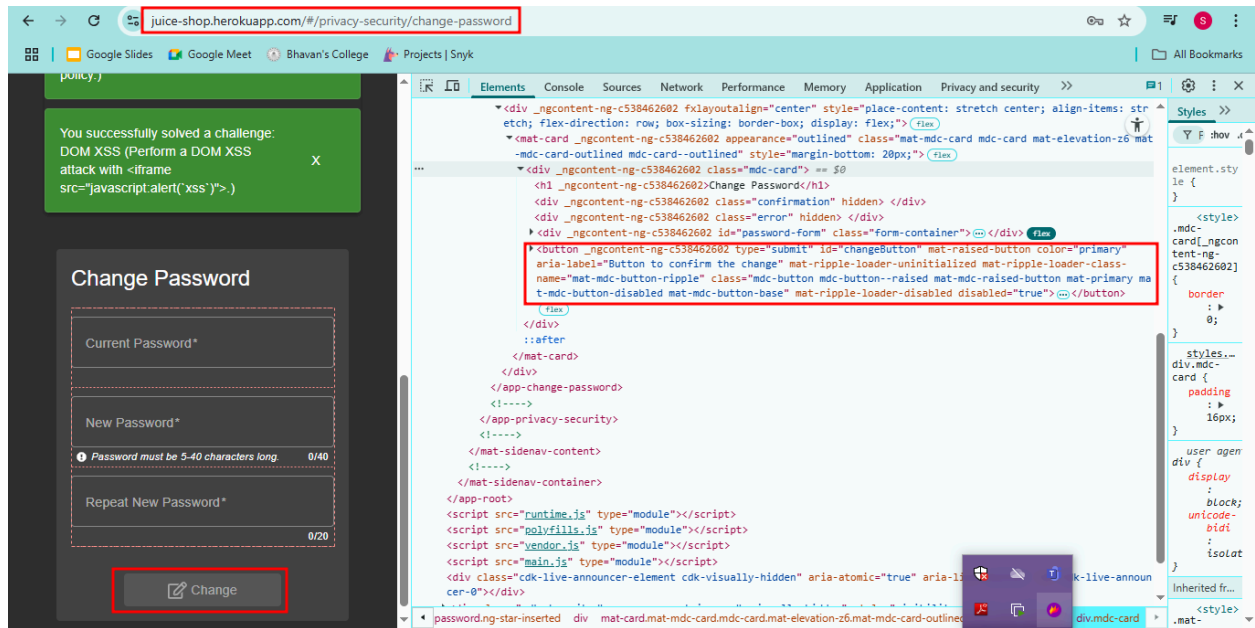
Affect on

Infected URL	https://juice-shop.herokuapp.com/#/privacy-security/change-password
Infected Parameter	No direct input parameter — vulnerability is triggered by bypassing the disabled attribute on the Change button in the HTML.
Parameter type	Form submission trigger (<button> element).
Attack vector	An attacker with access to a logged-in session can use browser Developer Tools to remove the disabled attribute from the “Change” button, bypassing the current password check and directly changing the account password.

- Severity:- High – Allows an attacker with access to a logged-in session to change the account password without knowing the current password.
- Ease of Exploitation :- Very Easy – Only requires using browser developer tools; no special skills or tools beyond standard web browser capabilities.

Analysis

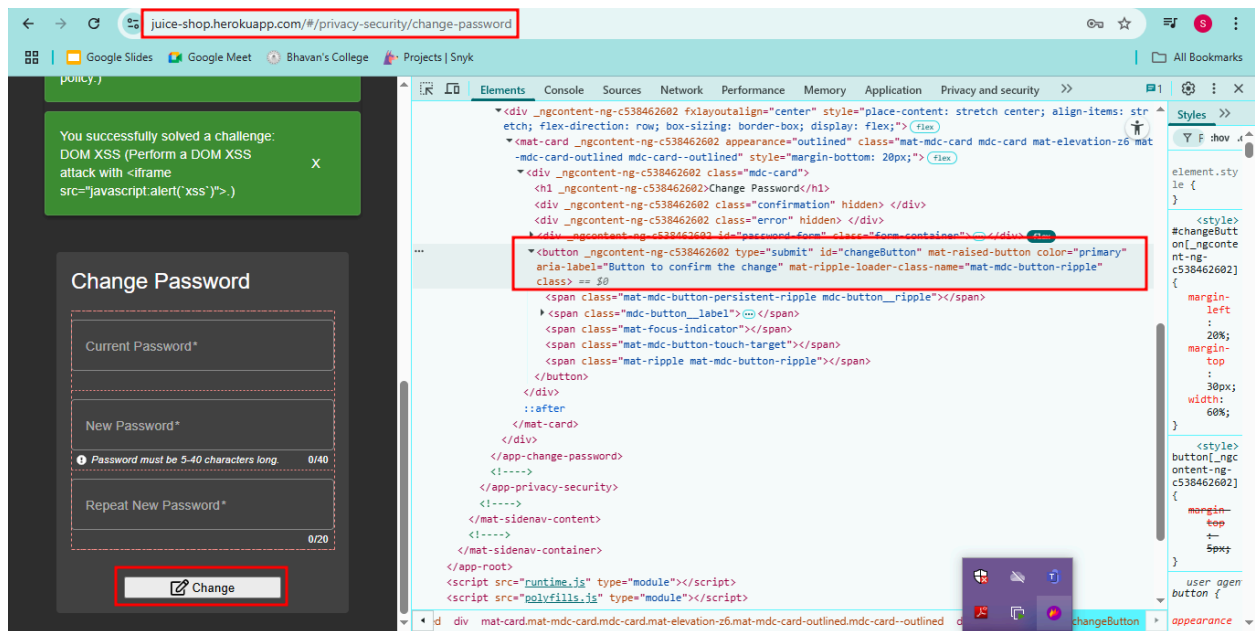
BEFORE



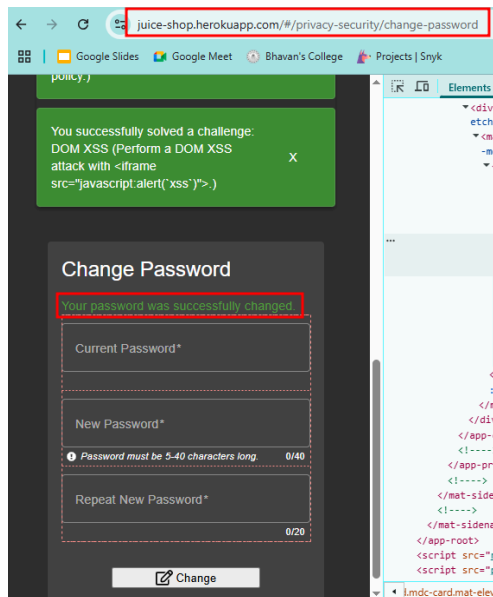
While inspecting the Change Password page code in the browser's Developer Tools, I observed that the Change button element had the `disabled="true"` attribute by default. This attribute was preventing the form from being submitted until all required fields were filled out, including the Current Password.

To test if this control could be bypassed, I manually edited the HTML code in the Elements panel.

AFETR



I removed the `disabled="true"`. After making this change, the "Change" button immediately became active. Without entering anything in the Current Password field, I entered matching values in New Password and Repeat New Password. I clicked the Change button.



The application successfully changed the password without requiring the current password.

Root Cause

- Absence of server-side verification of the current password before updating the password.
- Over-reliance on client-side controls (disabled button state) for enforcing security.

Impact

- An attacker with access to a victim's active session (e.g., via stolen cookies, unattended device, or shared terminal) can:
 - Change the victim's password.
 - Lock the victim out of their account.
 - Maintain long-term access by setting a password only they know.

Mitigation

1. Server-side validation:
 - Always verify the current password on the backend before processing a password change.
2. Ignore client-side state for security decisions — treat it as UX only.
3. Session re-authentication:
 - Require the user to re-enter credentials before performing sensitive actions like password changes.
4. Audit logging:
 - Record all password change events with timestamps and IP addresses for monitoring.

5. DOM XSS in Product Search

Affect on

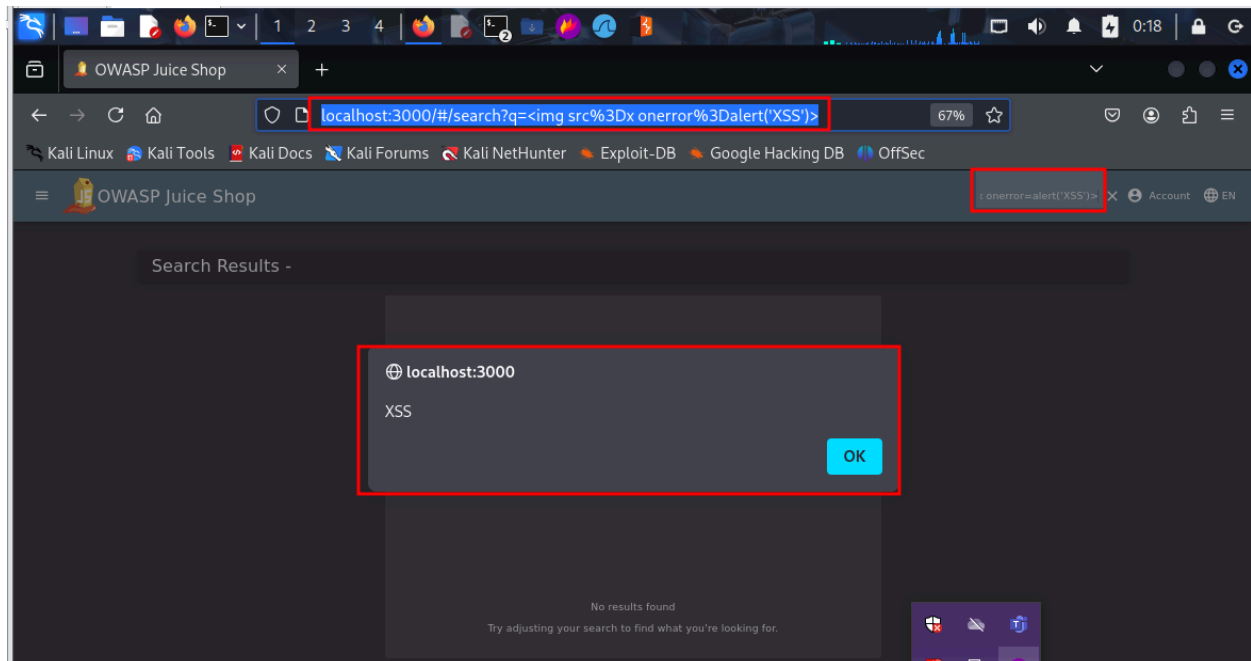
Infected URL	http://localhost:3000/#/search?q=%3Cimg%20src%3Dx%20onerror%3Dalert('XSS')%3E
Infected Parameter	q (search query)
Parameter type	URL query parameter
Attack vector	

- Severity: High – Because it can lead to full session hijacking, credential theft, or malicious actions in the user's browser.
- Ease of Exploitation: Medium

Analysis

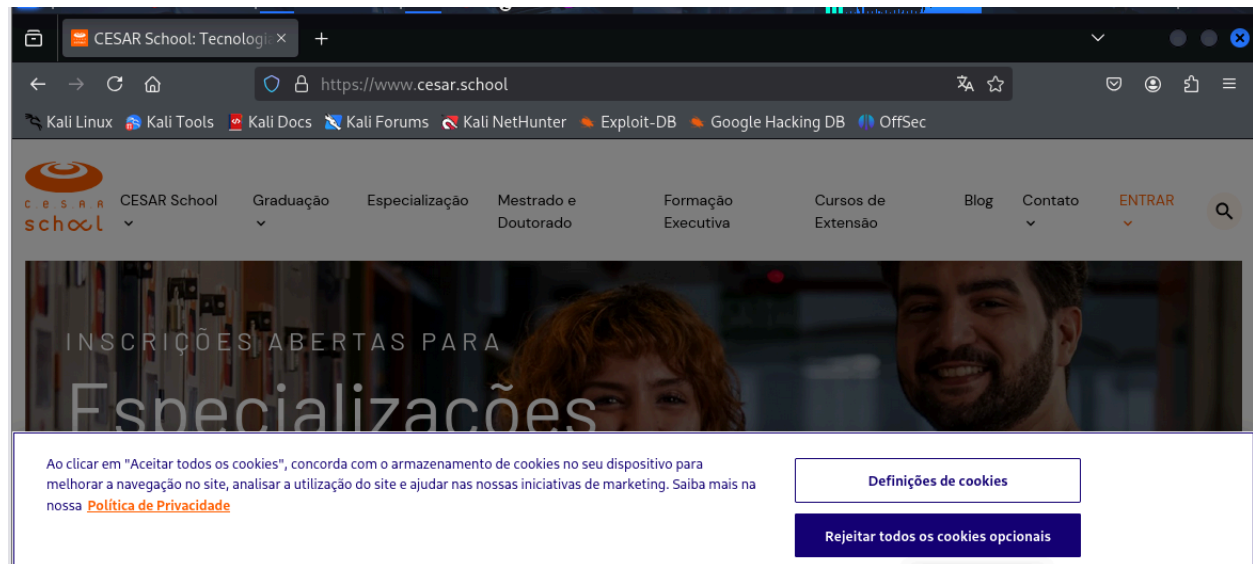
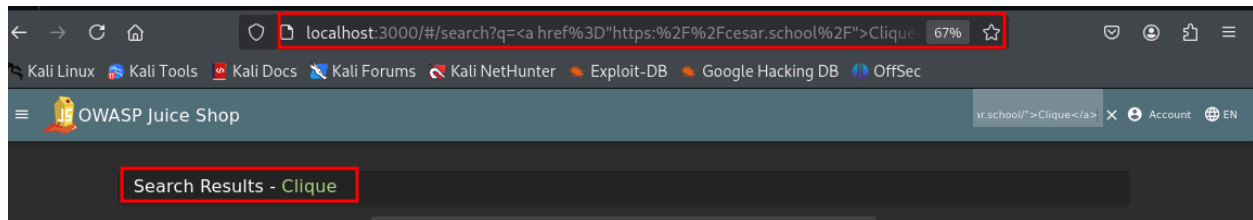
The product search functionality is vulnerable to DOM-based XSS. By entering the payload in the browser's search bar, the application executes the script in the context of the user's browser.

This payload triggered an alert box, demonstrating the presence of an XSS vulnerability.



This payload created a link that, when clicked, redirected the user to another page.

`<a href="https://cesar.school/">Clique</a>`



Got redirected to another page

Root Cause

- The application directly uses untrusted user input from the search query to manipulate the DOM without proper validation or sanitization.
- No output encoding is applied before rendering the input into HTML/JavaScript context.
- Lack of proper client-side and server-side filtering of special characters that can break HTML or JavaScript context.

Impact

- Execution of arbitrary JavaScript in the victim's browser.
- Theft of session cookies, tokens, or sensitive data.
- Possible redirection to malicious websites.
- Unauthorized actions performed on behalf of the victim (session hijacking or account takeover).

Mitigation

- Use proper output encoding before inserting user input into the DOM (e.g., HTML entity encoding).
- Avoid using innerHTML with untrusted data; use safe methods like `textContent` or `innerText`.
- Implement strict input validation and sanitization on both client and server sides.
- Use Content Security Policy (CSP) to reduce script execution risks.

6. Unauthorized Access to Administrative Panel

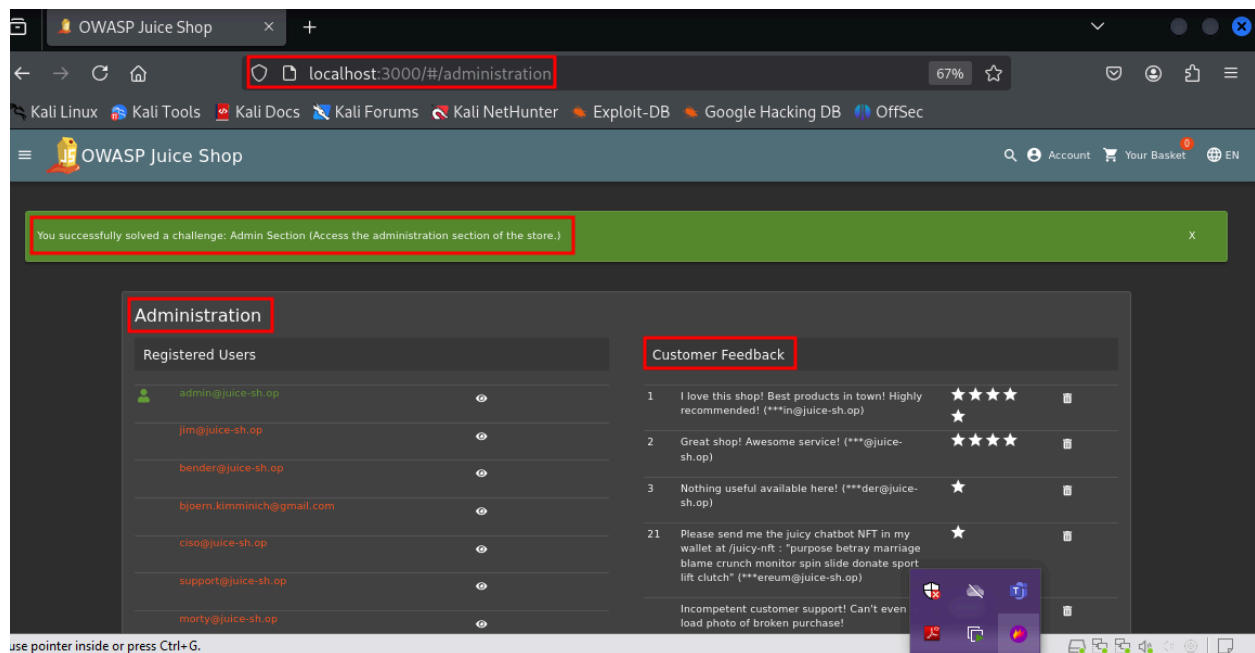
Affect on

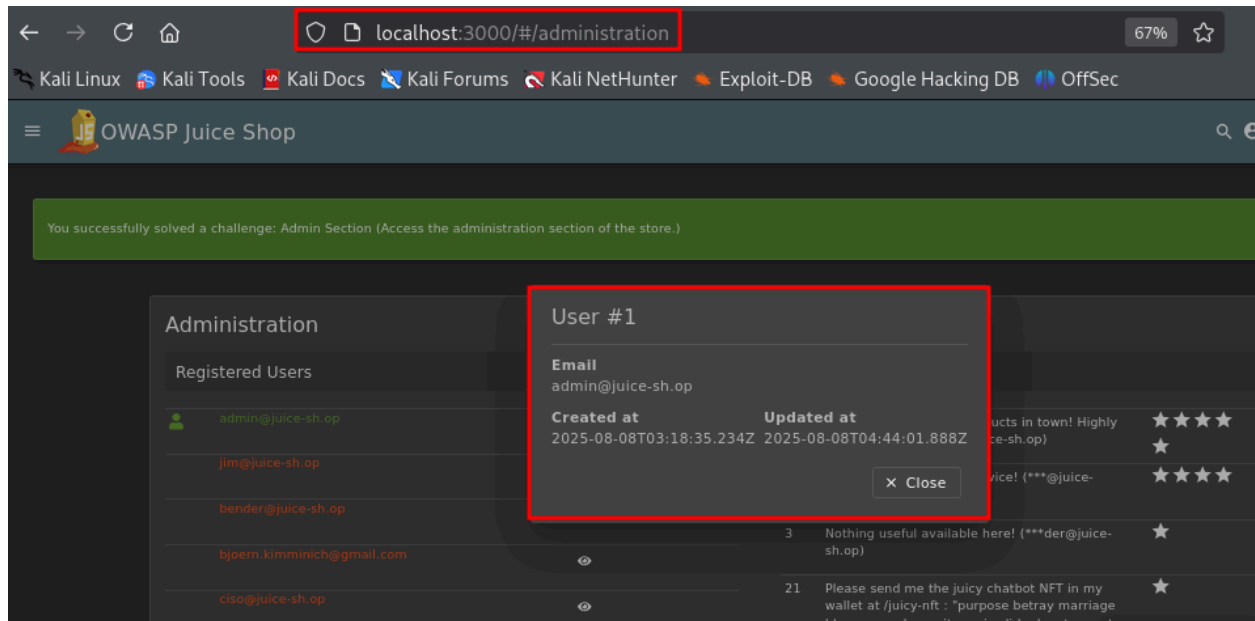
Infected URL	http://localhost:3000/#/administration
Infected Parameter	No direct parameter in the request — vulnerability lies in insufficient access control.
Parameter type	N/A (Access control flaw rather than parameter-based injection)
Attack vector	Direct access to /#/administration endpoint without proper authentication/authorization checks.

- Severity :- High
- Ease of Exploitation:- Easy – requires only knowledge or guessing of the URL path to access admin resources.

Analysis

During testing of the OWASP Juice Shop application, it was identified that the /administration route could be accessed directly without proper server-side authorization enforcement. By navigating directly to `http://localhost:3000/#/administration`, the application displayed the administrative dashboard containing a list of registered users along with their email addresses and customer feedback data.





sensitive details for all the User were accessible (above is the example for user #1):

- **Email:** admin@juice-sh.op
- **Created at:** 2025-08-08T03:18:35.234Z
- **Updated at:** 2025-08-08T04:44:01.888Z

Root Cause

- Lack of proper server-side authorization verification before rendering the admin page.
- The client-side route control is insufficient, allowing attackers to directly navigate to privileged routes.

Impact

- Exposure of sensitive user data (email addresses, customer feedback).
- Potential for account takeover or privilege escalation if combined with other vulnerabilities.
- Compromise of administrative functions leading to manipulation or deletion of records.

Mitigation

- Implement robust server-side authorization checks for all sensitive endpoints.
- Restrict access to admin routes based on user roles verified on the server.
- Do not rely solely on client-side route hiding or JavaScript-based access control.
- Employ proper session handling and role-based access control (RBAC).

7. Insecure Direct Object Reference

Affect on

Infected URL	http://localhost:3000/#/basket/1
Infected Parameter	BasketId
Parameter type	Path Parameter (numeric ID)
Attack vector	An attacker can modify the basketId value in the /rest/basket/{basketId} endpoint to access or modify another user's shopping basket without authorization checks

- Severity :- High – This vulnerability allows unauthorized access to another user's private shopping cart data, potentially leading to exposure of sensitive information or unauthorized transactions.
- Ease of Exploitation :- Easy – Can be exploited by simply intercepting the HTTP request (via Burp Suite, browser dev tools, or curl) and modifying the basket ID.

Analysis

The /rest/basket/{basketId} endpoint is vulnerable to an Insecure Direct Object Reference (IDOR). The basket ID is passed as a numeric path parameter in the API request, and the server does not verify that the requesting user is the legitimate owner of the basket.

In addition to viewing the basket contents, the API response also revealed the `userId` of the basket owner. This information disclosure increases the overall impact of the vulnerability, as it could be used in further targeted attacks or chained with other vulnerabilities.

Root Cause

- Lack of authorization check on the `/rest/basket/{basketId}` endpoint.
- Use of direct, predictable object identifiers in API requests.
- Failure to enforce ownership validation before returning basket details.

Impact

- Exposure of other users' basket contents.
- Potential to alter another user's basket by adding/removing items.
- Could be chained with other vulnerabilities (e.g., price manipulation or unauthorized purchases).
- Privacy violations and possible fraud.

Mitigations

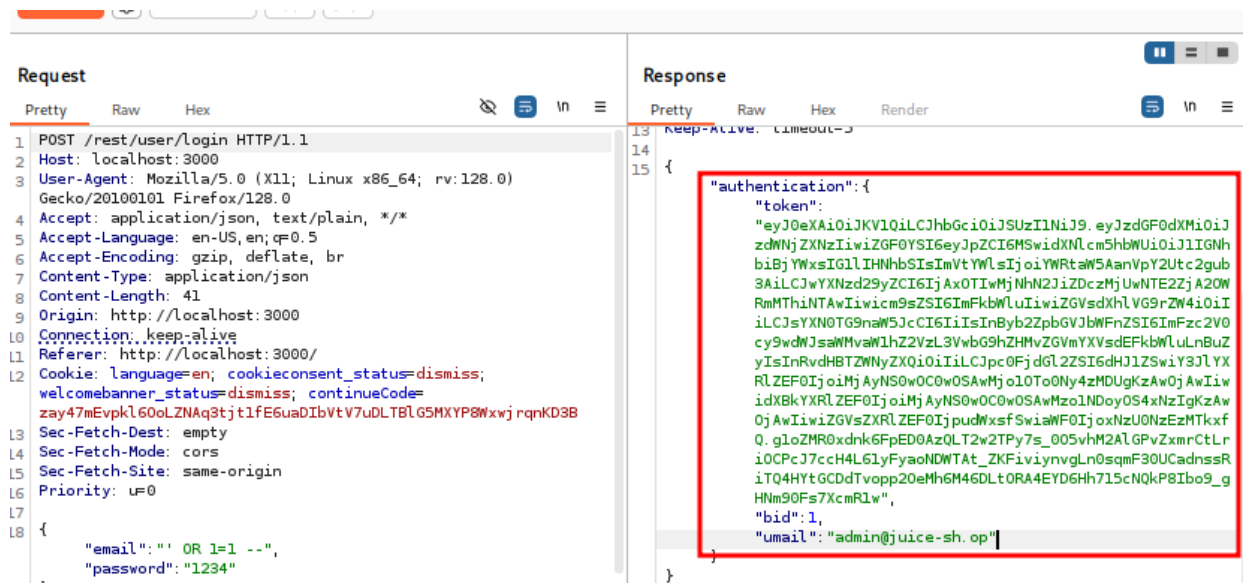
- Implement Ownership Checks
Verify that the `basketId` requested belongs to the currently authenticated user before returning or modifying data.
- Use Indirect Object References
Replace numeric IDs with UUIDs or opaque tokens that are harder to guess.
- Apply Access Control at the API Level
Enforce proper authorization logic for every request, not just on the UI side.
- Regular Security Testing
Include Broken Access Control/IDOR checks in regular penetration tests and automated security scans.

Affect on

- **Severity :- High** – Session cookies are critical for authentication, and their theft can completely compromise an account.
- **Ease of Exploitation :- Easy** – Exploitation only requires an XSS or other JavaScript injection point, which is common in web apps.

Analysis

During testing, it was observed that the application's authentication cookie (token) is set without the `HttpOnly` attribute.



Since the `HttpOnly` flag is missing, the cookie is accessible to JavaScript running in the browser context. If an attacker successfully injects malicious JavaScript into the application (via XSS), they can read and exfiltrate this cookie.

Root Cause

The session cookie is generated and sent to the client without explicitly setting the **HttpOnly** attribute in the server's response headers

Impact

- Session Hijacking – Attackers can take over valid user sessions.
- Account Compromise – User accounts, including admin accounts, can be accessed.
- Privilege Escalation – If an admin session is stolen, the attacker can control the entire application.
- Data Theft – Sensitive user data could be exposed.

Mitigation

- Set HttpOnly Flag – Modify cookie creation code to include HttpOnly.
- Implement Content Security Policy (CSP) – Reduce the likelihood of XSS.
- Validate and sanitize all inputs – Prevent injection points that allow JavaScript execution.
- Use Secure Flag with HTTPS – Ensure cookies are only sent over secure channels.

9. XML External Entity (XXE)

Affect on

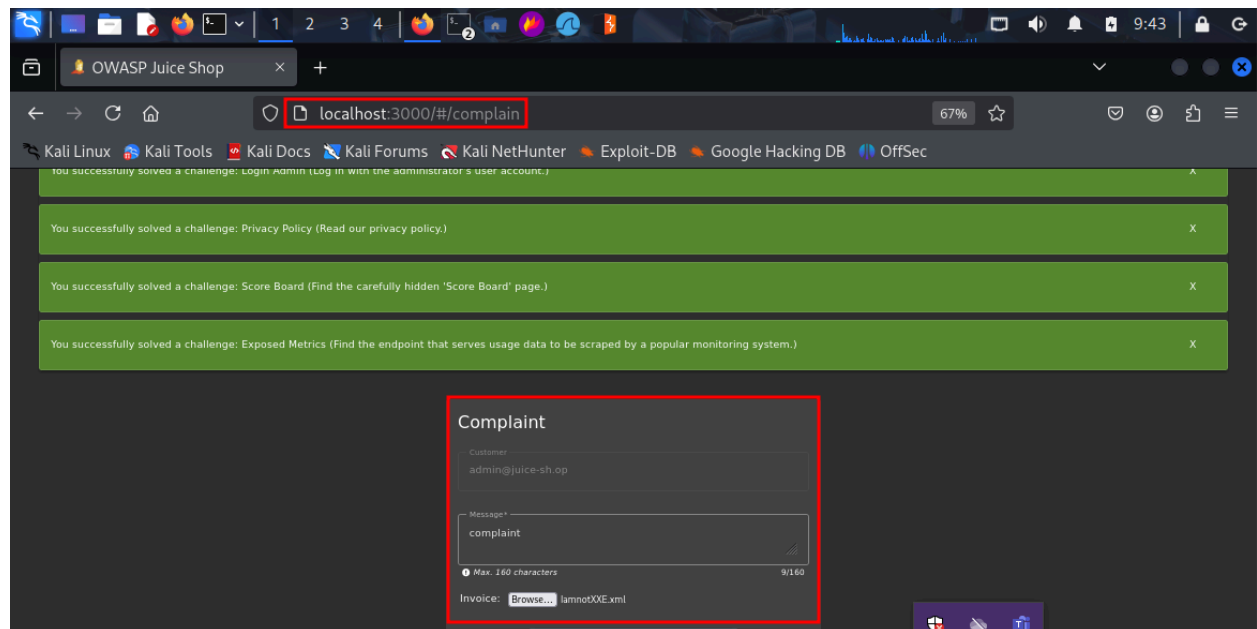
Infected URL	http://localhost:3000/#/complain
Infected Parameter	[Parameter name that accepts the XML input, e.g., "xml_data", "payload", or the request body if raw XML]
Parameter type	XML input / POST body
Attack vector	I injected a malicious XML payload containing an external entity definition that references a local system file.

- Severity:- High
- Ease of Exploitation:- High

Analysis

An XML payload was crafted with the following structure

```
<?xmlversion="1.0"encoding="UTF-8"?>
<!DOCTYPEfoo[<!ELEMENTfooANY>
<!ENTITYxxeSYSTEM"file:///C:/Windows/system.ini">]>
<trades>
<metadata>
<name>SecureIdeas</name>
<trader>
<foo>&xxe;</foo>
<name>K.Johnson</name>
</trader>
<units>24</units>
<price>13.99</price>
<name>Picanha</name>
<trader>
<name>JasonGillam</name>
</trader>
<units>CAD</units>
<price>buck</price>
</metadata>
</trades>
```



```
"file:///C:/Windows/system.ini"><trades><meta  
<name>Secure Ideas</name><trader><foo>;  
for 16-bit app  
support[386Enh]woafont=dosapp.fonEGA80WOA.FON=EGA80WOA.FONEGA40WOA.FON=EG  
A40WOA.FONCGA80WOA.FON=CGA80WOA.FONCGA40WOA.FON=CGA40WOA.FON[drivers]wave  
=mmdrv.dlltimer=timer.drv[mci]</foo><name>K. Johnson</n...  
(ImNotXXE.xml)
```

When this file was uploaded, the server parsed it and resolved the external entity &xxe;, reading the contents of C:/Windows/system.ini. The sensitive content from this file appeared in the server's response.

Root Cause:

The XML parser used by the application processes external entity definitions without proper restrictions, allowing attackers to define and include external entities referencing local files or network resources. This indicates a lack of secure XML parser configuration or validation.

Impact:

- **Information Disclosure:** Sensitive files on the server (e.g., system configuration files) can be read and disclosed to an attacker.
- **Potential further attacks:** Depending on server setup, XXE can be leveraged to perform SSRF, Denial of Service, or Remote Code Execution.
- **Data leakage risk:** Disclosure of system internals can assist attackers in further exploitation.

Mitigations:

1. **Disable External Entity Processing:** Configure XML parsers to disable DTDs and external entities. For example, in Java, use features like:
 - `factory.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);`
 - `factory.setFeature("http://xml.org/sax/features/external-general-entities", false);`

- `factory.setFeature("http://xml.org/sax/features/external-parameter-entities", false);`
- 2. Use Less Complex Data Formats: When possible, avoid XML in favor of safer data formats like JSON.
- 3. Input Validation: Strictly validate and sanitize all XML inputs before parsing.
- 4. Apply Security Controls: Use web application firewalls (WAF) that detect and block XXE payloads.
- 5. Keep Libraries Updated: Ensure XML parsing libraries are up-to-date with security patches.

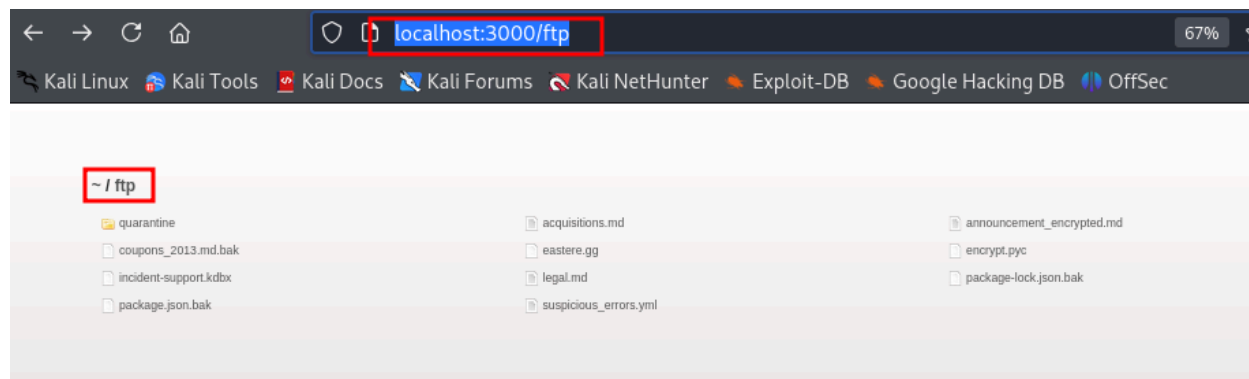
10. File Path Traversal with Information Disclosure via Improper Error Handling

Affect on

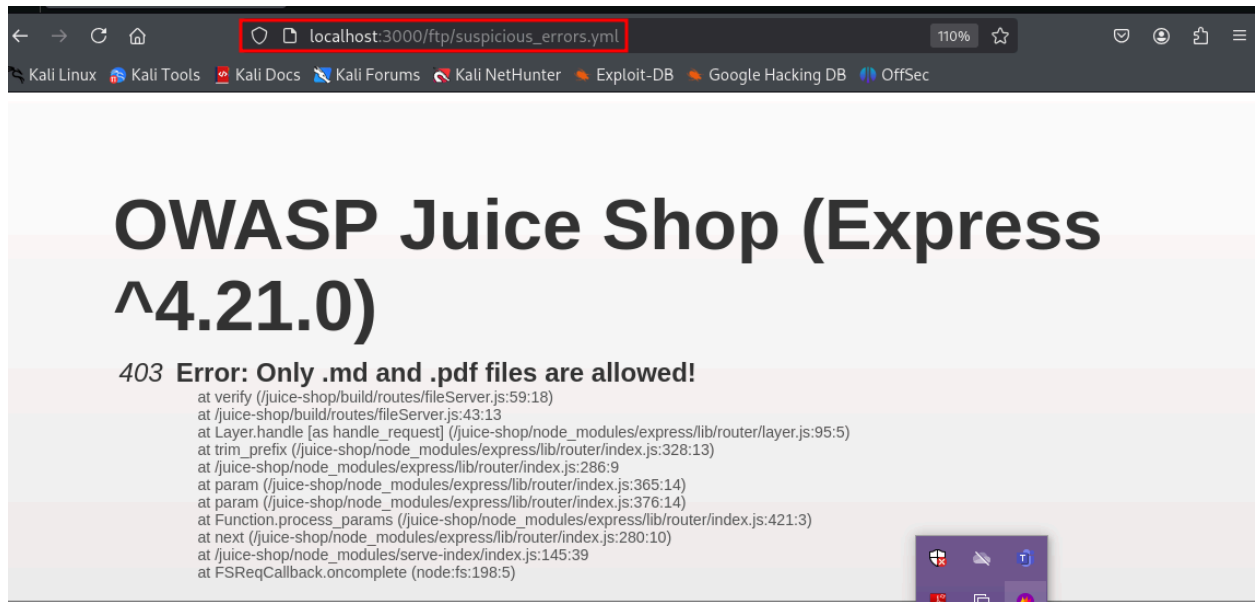
Infected URL	● http://localhost:3000/ftp ● http://localhost:3000/ftp/suspicious_errors.yml
Infected Parameter	● /ftp/ directory path ● File name in the URL path
Parameter type	Path parameter (URL directory path).
Attack vector	An attacker directly accesses the /ftp/ directory via the browser or automated tools, exploiting enabled directory listing to enumerate and download sensitive files without authentication.

- Severity: High
- Ease of exploitation :- Easy

The /ftp/ directory in OWASP Juice Shop has directory listing enabled, allowing unauthenticated users to view the complete list of files stored in that location. This exposes potentially sensitive files, including backups (.bak), configuration data, and other non-public resources.



During testing, the application's file retrieval functionality was found to be vulnerable to **File Path Traversal** when accessing files under /ftp/. Supplying a file name not permitted by the application's whitelist (e.g., suspicious_errors.yml) triggered a verbose error message instead of a sanitized response.



This reveals:

- The framework in use (**Express** and its version)
- File system paths (e.g., `/juice-shop/build/routes/fileServer.js`)
- Internal application logic and routing structure

You successfully solved a challenge: Error Handling (Provoke an error that is neither very gracefully nor consistently handled.)

X

Impact

- Disclosure of sensitive files (`/etc/passwd`, configuration files, logs)
- Information about server-side code structure and file locations

- Leverage for further attacks (RCE, LFI → RCE via log poisoning, etc.)

Root Cause

- No path sanitization: Application doesn't remove ../ sequences from user input.
- Verbose error messages: Server returns detailed error output instead of generic messages.

Mitigation

1. Input validation & sanitization
 - Allow only whitelisted filenames (e.g., from a database).
 - Strip or reject ../ and other traversal sequences.
2. Error handling
 - Show only generic error messages to users.
 - Log detailed errors internally.
3. Least privilege
 - Restrict the app's file access to only the necessary directories.

11. Business Logic Vulnerability – Bypass of Deluxe Membership Payment

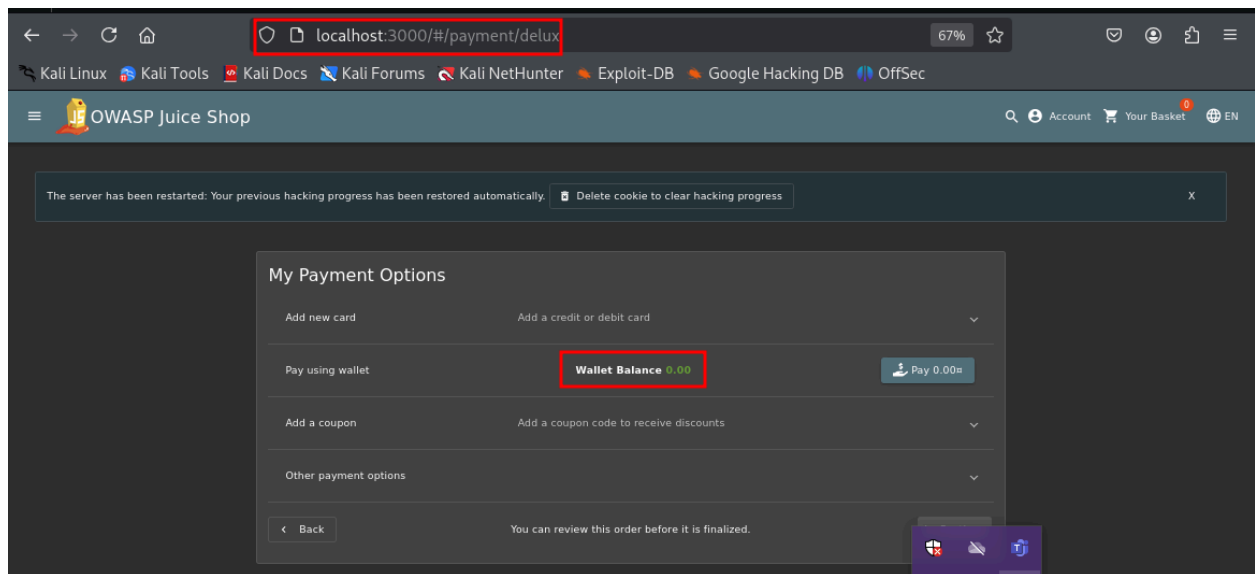
Affect on

Infected URL	http://localhost:3000/#/payment/delux
Infected Parameter	Membership
Parameter type	Client-side storage API request parameter
Attack vector	Modifying client-side data (like localStorage or cookies) or intercepting API requests to change the membership status to “deluxe,” exploiting the lack of proper server-side validation.

- Severity: High — This vulnerability grants unauthorized access to premium features, leading to direct revenue loss and breach of business logic.
- Ease of Exploitation: Easy — Requires minimal technical skills such as using browser developer tools or proxy interceptors. No complex cryptographic or injection skills needed.

Analysis

Initially, when browsing to <http://localhost:3000/#/payment/deluxe> the payment options were displayed correctly, with a noted account balance of \$0 in the wallet, as seen in the accompanying screenshot.



Through an examination of the web page using developer tools, Secure Ideas observed that the payment button attribute was set to "disabled=true", suggesting that the system was correctly disallowing payment attempts due to the absence of funds in the wallet. The relevant code snippet and the corresponding screenshot are included below.

```

width: 30%;>>></div>
<div _ngcontent-bfm-cl00 flex="20%" style="flex: 1 1 100%; box-sizing: border-box; max-
width: 20%;">
  <button _ngcontent-bfm-cl00 type="submit" color="primary" mat-raised-button class="mat-
focus-indicator btm mat-raised-button mat-button-base mat-primary mat-button-disabled"
style="float: right;" disabled="true">
  <span class="mat-button-wrapper">
    <svg _ngcontent-bfm-cl00 class="svg-inline--fa fa-hand-holding-usd fa-w-18 fa-lg"
aria-hidden="true" focusable="false" data-prefix="fas" data-icon="hand-holding-usd"
role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>
    </svg>
    <!-- <i _ngcontent-bfm-cl00="" class="fas fa-hand-holding-usd fa-lg"></i> Font
Awesome fontawesome.com -->
    " Pay 49.00x " == $0
  
```

Next, the "disabled" attribute from the payment button was manually removed, which enabled the payment button to be clicked. As a result, a POST request was initiated, which predictably returned an error message, "Insufficient funds in wallet". This behavior is appropriate as the "paymentMode" was set to "wallet". Subsequently, the request was captured using the Burp Suite Repeater tool. In this intercepted request, the "paymentMode" parameter was manipulated by replacing "wallet" with an empty string, and then the request was resent. This resulted in a successful upgrade to the deluxe membership, confirmed by the application's response- "Congratulations! You are now a deluxe member!" Despite the wallet balance being zero.

The OWASP Juice Shop application allowed the account upgrade, indicating a vulnerability in the payment processing logic. This successful response is displayed in the screenshot below.

```

{
  "status": "success",
  "data": {
    "confirmation": "Congratulations! You are now a deluxe member!",
    "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXRhbnZlbnZlZGF0YSI6eyJpZC...
  }
}

```

Root Cause:

The application trusts client-side data or fails to properly verify membership status on the server. Lack of proper server-side validation or authorization checks allows attackers to escalate privileges by modifying data that should be immutable or verified.

Impact:

- Unauthorized access to premium features and content reserved for Deluxe members.
- Potential loss of revenue due to users bypassing payment requirements.
- Breach of trust and compromise of user privileges.

Mitigations:

1. Enforce Server-Side Validation: All membership or role changes must be validated and authorized by the server only. Never trust client-side data for critical access control.
2. Use Signed Tokens: If membership info is stored in tokens (like JWT), ensure tokens are signed and validated. Do not accept client modifications without verification.
3. Secure API Endpoints: Implement strict authorization checks on endpoints handling membership upgrades or changes.
4. Data Integrity Checks: Use tamper-proof mechanisms to ensure membership data cannot be altered by clients.

12. Insecure Transmission of Credentials via HTTP

Affect on

Infected URL	http://localhost:3000/#/search
Infected Parameter	username password
Parameter type	Text input (form field)
Attack vector	If HTTP is exposed beyond localhost, an attacker on the network could sniff credentials using a MITM attack (e.g., ARP spoofing, Wi-Fi sniffing).

Severity:- High

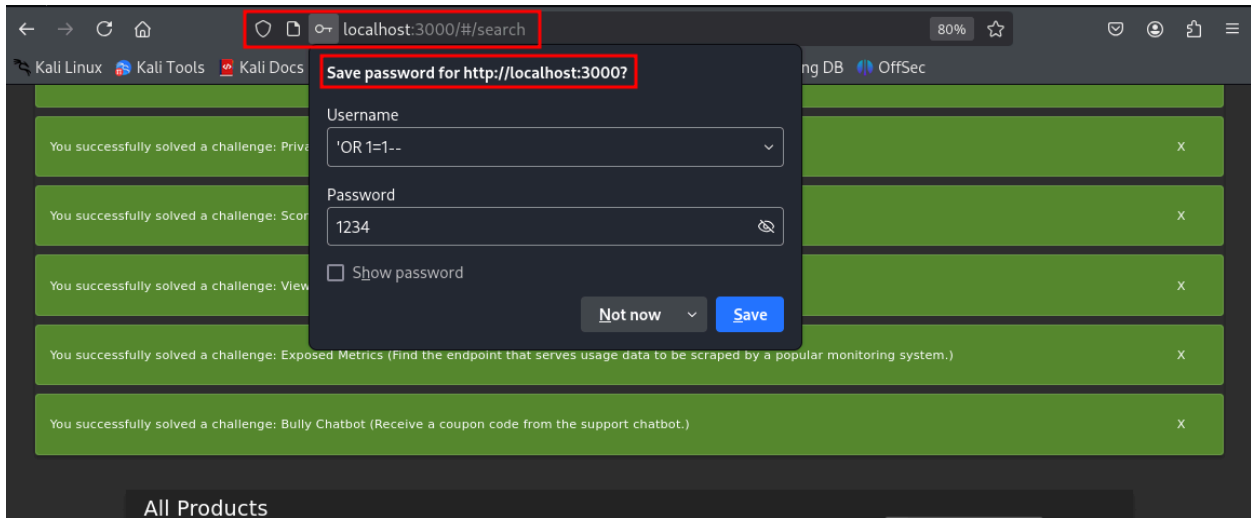
Ease of Exploitation:

- Local attacker: Very easy — modern browsers store passwords in readable form if the user is logged into their OS/browser profile.
- Network attacker: Easy with tools like Wireshark or mitmproxy if the traffic is not encrypted.

Analysis

The application uses plain HTTP instead of HTTPS, allowing credentials to be transmitted without encryption. Additionally, the login form does not disable browser password auto-save (autocomplete="off" is not implemented). This combination increases the attack surface:

- Network-level attackers can intercept credentials in transit.
- Local attackers can retrieve saved credentials from the browser's password manager.



Root Cause

- No TLS/HTTPS configuration for the login endpoint.
- No HTML attribute to discourage browsers from storing sensitive credentials.

Impact

- Compromise of user accounts.
- Privilege escalation if stolen credentials belong to administrative accounts.
- Potential lateral movement if credentials are reused across other systems.

Mitigation

1. Implement HTTPS — Use TLS certificates to encrypt traffic and prevent MITM interception.
2. Set autocomplete="off" for username and password fields to discourage browsers from storing credentials.
3. Use Secure and HttpOnly flags for session cookies.
4. Restrict physical and remote access to client machines running the application.
5. Encourage users to use password managers with encryption and master passwords.

13. Outdated JavaScript Library – jQuery v2.2.4

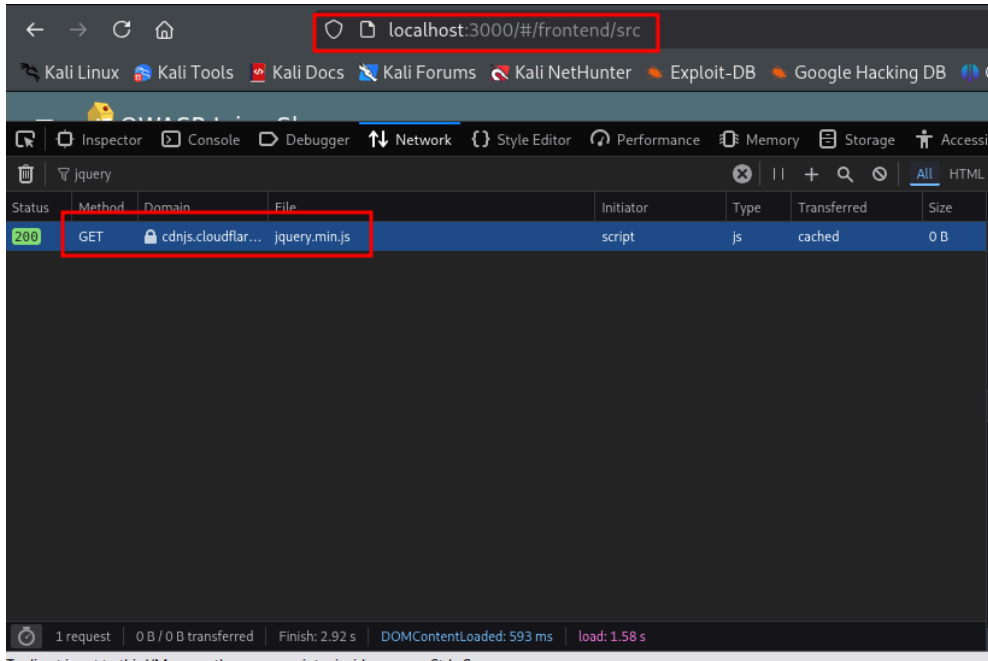
Affect on

Infected URL	https://cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js
Infected Parameter	Not applicable — this is a static JavaScript resource, not a user-input parameter.
Parameter type	Static JavaScript library file (client-side code).
Attack vector	The application loads jQuery v2.2.4, which contains publicly known vulnerabilities including: • CVE-2015-9251 – Cross-Site Scripting (XSS) via jQuery.html() and jQuery.append() methods. • CVE-2019-11358 – Prototype pollution via jQuery.extend(true, {}, ...) in certain versions.

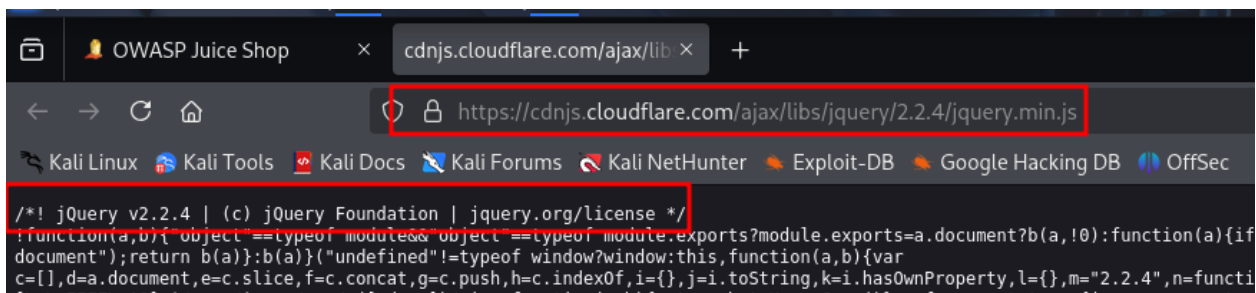
- Severity: Medium– High – Severity depends on whether vulnerable jQuery functions are called with user-supplied data. Since this is a known insecure version with multiple public exploits, it's considered high risk in security audits.
- Ease of Exploitation: Moderate – The vulnerability can be exploited without authentication if certain vulnerable functions are exposed in the UI. Attackers can test and inject payloads via reflected/stored input points.

Analysis

While performing an authenticated and unauthenticated crawl of the OWASP Juice Shop web application, all loaded scripts were captured using the browser's **Developer Tools** → **Network tab**. Several JavaScript files were observed being loaded from the application's root, including a large bundled file named min.js



Inspecting the main.js file (via Developer Tools) revealed that it contains a copy of the **jQuery library** embedded within the bundle.



This confirmed that the application is using jQuery v2.2.4.

jQuery v2.2.4 was cross-checked against the public CVE database and jQuery Security Advisories, revealing multiple known vulnerabilities

The screenshot shows the CVE Details page for jQuery 2.2.4. The URL in the browser is `cvedetails.com/vulnerabilities/11581010/jQuery-jQuery-2.2.4-PENT`. The page title is "jQuery » jQuery » 2.2.4". Under "Version names", there are three entries: "jQuery 2.2.4", "opencs/jQuery/jQuery/2.2.4", and "opencs/jQuery/jQuery/2.2.4". Under "Product information", there are two entries: "https://github.com/jquery/jquery/ Version" and "http://jquery.com/ Vendor". A table titled "Vulnerabilities by types/categories" shows the following data:

Year	Overview	Memory Corruption	SQL Injection	XSS	Directory Traversal	File Inclusion	CSRF	XOR	SSRF	Open Redirect	Input Validation
2018	0	0	0	1	0	0	0	0	0	0	0
2019	0	0	0	0	0	0	0	0	0	0	0
2020	0	0	0	1	0	0	0	0	0	0	0
2021	0	0	0	1	0	0	0	0	0	0	0
Total				3							

Because the vulnerable version is loaded on every page, any DOM manipulation of untrusted data using insecure jQuery functions could be leveraged for **client-side attacks** such as XSS or prototype pollution.

Root Cause:

The project has not upgraded the jQuery dependency in its front-end build pipeline, leading to the continued use of an outdated and vulnerable version.

Impact:

If exploited, an attacker could:

- Execute arbitrary JavaScript in the victim's browser (XSS).
- Modify JavaScript object prototypes (prototype pollution), potentially affecting logic and security checks.
- Steal sensitive data or perform actions on behalf of the user (session hijacking, phishing).

Mitigation:

- Upgrade jQuery to the latest stable version (≥3.6.4 at the time of writing).
- Perform regular dependency audits using tools like npm audit or yarn audit.
- Apply Content Security Policy (CSP) headers to mitigate the impact of potential XSS.
- Avoid dangerous DOM manipulation functions (.html(), .append(), .before()) with untrusted input.

14. Broken Access Control: Bid Value Manipulation in Auction Feature

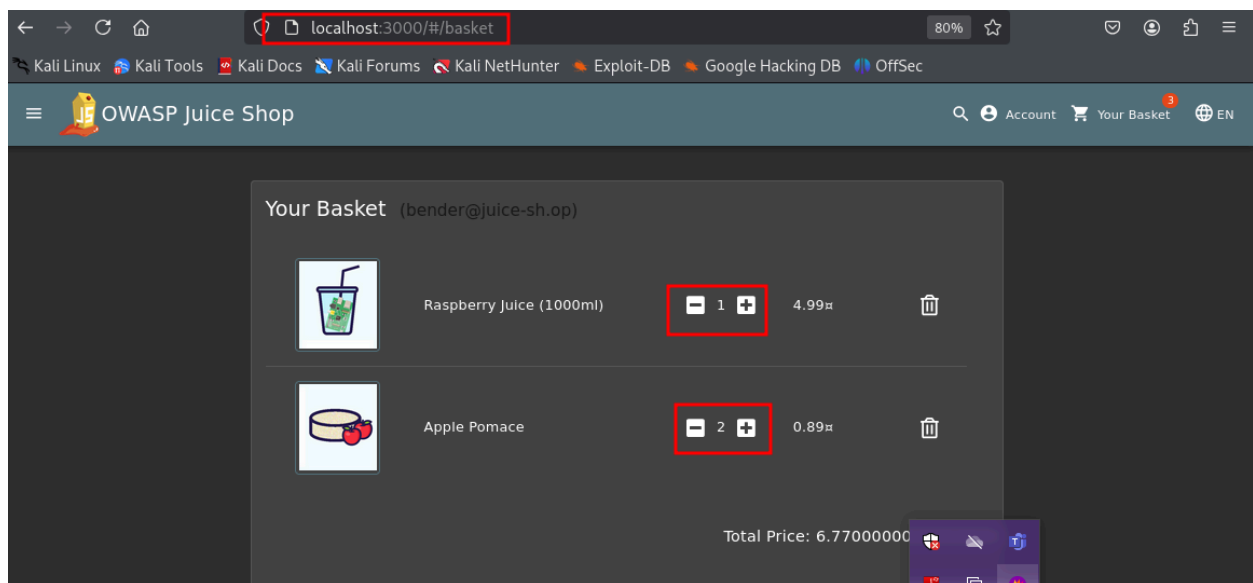
Affect on

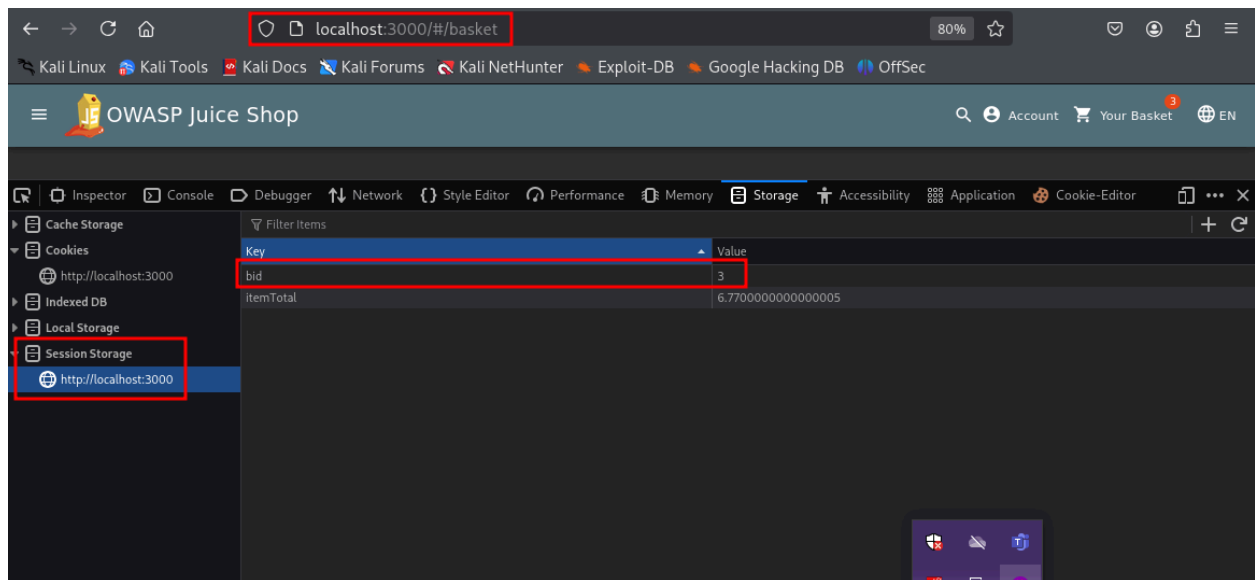
Infected URL	http://localhost:3000/#/basket
Infected Parameter	bid (Bid amount in the request payload)
Parameter type	Post
Attack vector	Intercept and modify the bid value in the POST request payload to submit unauthorized bids, bypassing server-side validation.

- Severity: High
- Ease of Exploitation: Moderate — requires basic knowledge of intercepting and modifying HTTP requests.

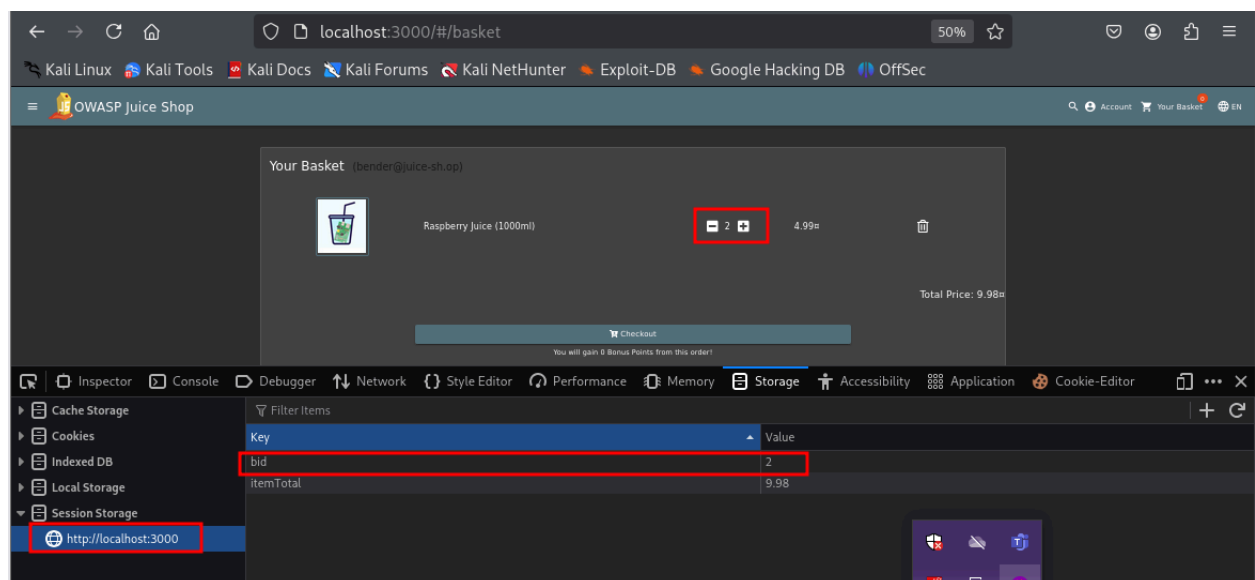
Analysis

Using browser developer tools, I intercepted the POST request sent when placing a bid.





I modified the bid parameter in the JSON payload to an unauthorized value lower than the current highest bid



Upon resubmitting the request, the server accepted the manipulated bid due to lack of proper validation.

Root Cause:

The application lacks proper server-side validation and authorization checks on the bid value submitted by the client, allowing manipulated bids to bypass business rules and be accepted.

Impact:

- Attackers can manipulate auction outcomes, leading to unfair advantage and loss of trust.
- Legitimate bidders may be disadvantaged or excluded.
- Overall compromise of auction integrity.

Mitigation:

- Implement strict server-side validation to ensure every new bid is higher than the current highest bid before accepting it.
- Do not rely solely on client-side checks; always validate and enforce business rules on the server.
- Use authentication and authorization controls to verify that the user placing the bid has permission to do so.
- Implement logging and monitoring to detect suspicious bidding activity.
- Consider rate limiting and CAPTCHA to prevent automated bid manipulation.

15. Reflected Cross-Site Scripting (XSS) in Order Tracking Input

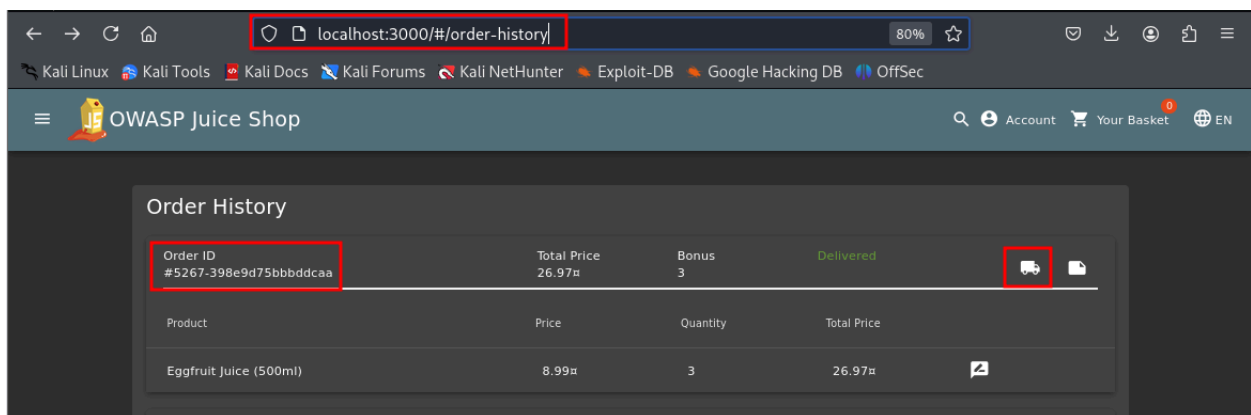
Affect on

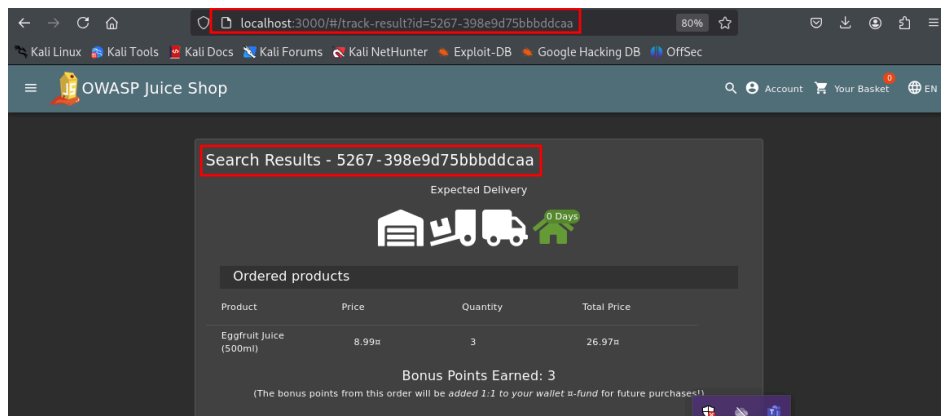
Infected URL	http://localhost:3000/#/track-result?id=%3Ciframe%20src%3D%22javascript:alert(%60data%20all%20over%20this%20screen%20that%20wasnt%20planned%60)%22%3E
Infected Parameter	orderId
Parameter type	Query parameter
Attack vector	Injecting a malicious script in the orderId query parameter causes the script to be reflected and executed in the browser, exploiting insufficient input sanitization.

- Severity :- High
- Ease of Exploitation :- Easy

Analysis

While testing the application, I navigated to the **Order History** page and clicked on **Track Order** to enter an order ID for tracking

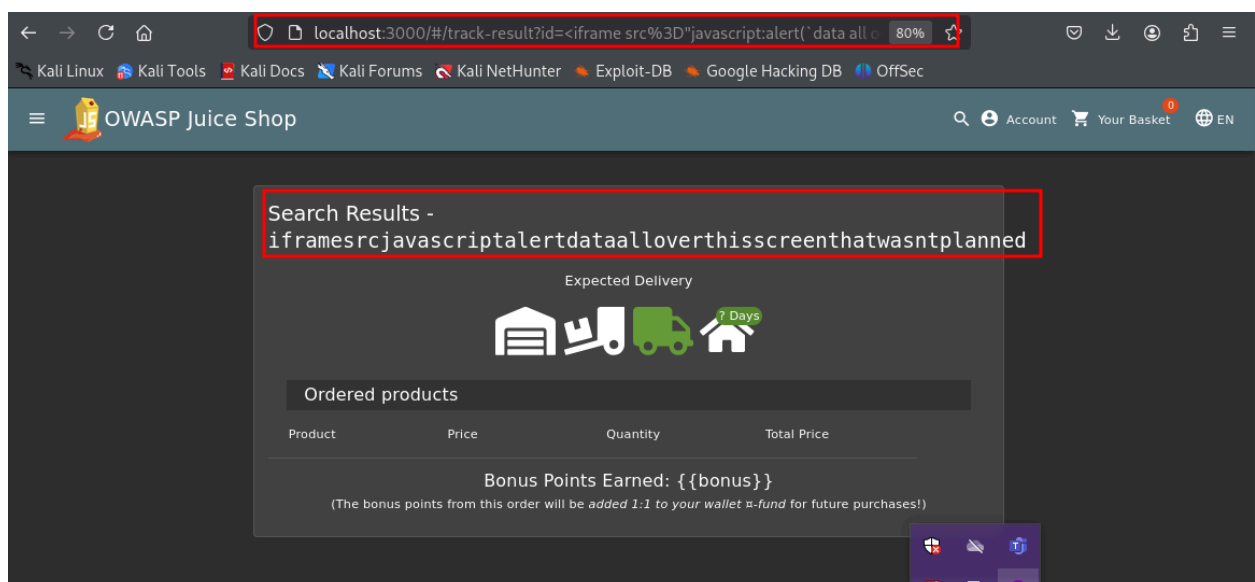




In the URL for the order ID, I injected a crafted payload: `<iframe src="javascript:alert(`data all over this screen that wasnt planned`)">`

URL-encoded as:

`%3Ciframe%20src%3D%22javascript:alert(%60data%20all%20over%20this%20screen%20that%20wasnt%20planned%60)%22%3E`



Upon submitting this payload, the application reflected the input without proper sanitization or encoding, causing the malicious script within the iframe to execute in the browser. This confirmed the presence of a **Reflected Cross-Site Scripting (XSS)** vulnerability in the orderId parameter of the order tracking functionality.

Root Cause

- The application reflects user input from the orderId query parameter into the page without proper sanitization or encoding.
- Lack of input validation and output encoding allows injection of executable scripts.

Impact

- Execution of arbitrary JavaScript in victims' browsers.
- Possible theft of session cookies, account hijacking, or phishing attacks.
- Damages user trust and application integrity.

Mitigation

- Sanitize and encode all user-supplied input before including it in HTML responses.
- Use context-aware output encoding (HTML, JavaScript, URL, etc.).
- Implement Content Security Policy (CSP) headers to restrict script execution.
- Use secure development frameworks and libraries that escape output by default.

16. Business Logic Error - Multiple Discount Request via Chat Box

Affect on

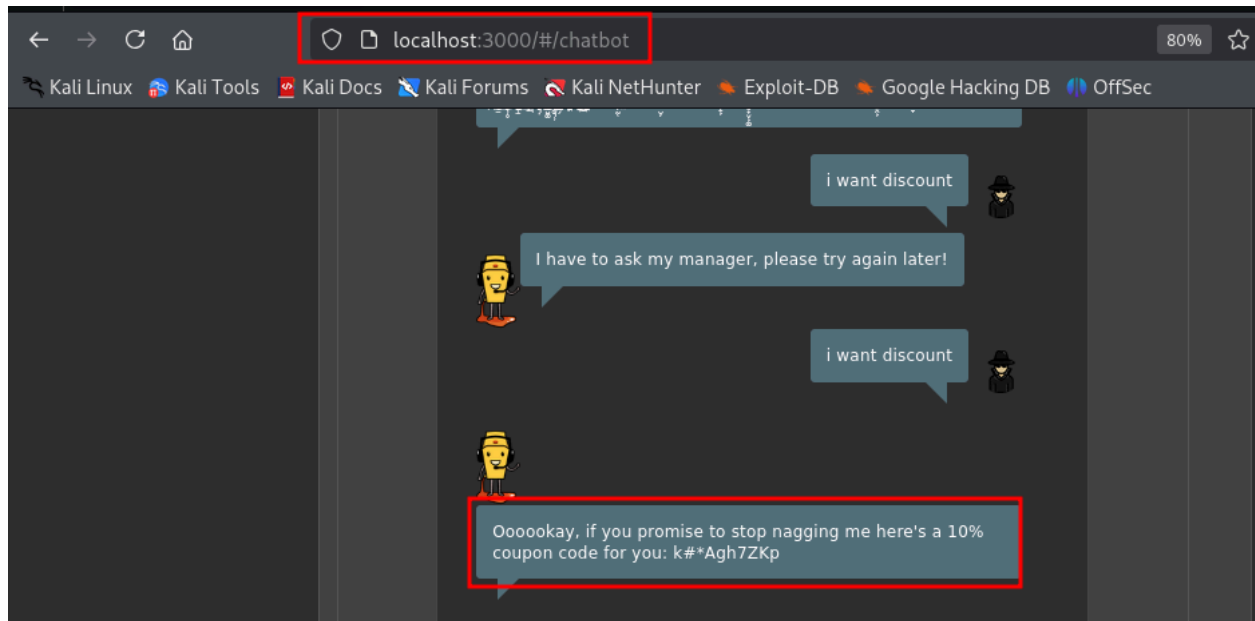
Infected URL	http://localhost:3000/#/chatbot
Infected Parameter	Message
Parameter type	Post
Attack vector	Excessive discount requests via the chatbox trigger a logic flaw, granting an unintended 10% discount without proper rate limiting or request validation.

- **Severity:- Medium** – The vulnerability allows unauthorized access to discount codes, leading to potential revenue loss.
- **Ease of Exploitation :- Easy** – No authentication bypass or advanced skills required. Can be done via the front-end chat UI or automated scripts (e.g., Burp Repeater).

Analysis

The chat bot is implemented with logic to respond to certain keywords or phrases. When I repeatedly ask for discounts, the system eventually rewards me with a discount code.

There is no rate limiting, per-user restriction, or validation to ensure only eligible users receive a discount. This leads to **business logic abuse**, the system's intended reward feature can be exploited in an unintended way.



You successfully solved a challenge: Bully Chatbot (Receive a coupon code from the support chatbot.)

Root Cause:

- Weak business logic validation in the chat bot code.
- Lack of server-side restrictions (no check for number of discounts per account, per day, or per session).
- The discount generation is purely trigger-based without proper authentication or verification steps.

Impact:

- Revenue Loss – Attackers can generate unlimited discount codes and share them with others.
- Inventory Abuse – Multiple discounts can encourage mass purchases at a loss.
- Reputation Damage – Public awareness of such a flaw can harm brand trust.

Mitigation:

- Implement rate limiting on discount-related requests in the chat bot.
- Add per-user discount limits (e.g., one per account/email/phone number per month).
- Require user authentication before issuing any discount code.
- Add CAPTCHA to prevent automated exploitation.
- Validate discount eligibility server-side before applying it at checkout.

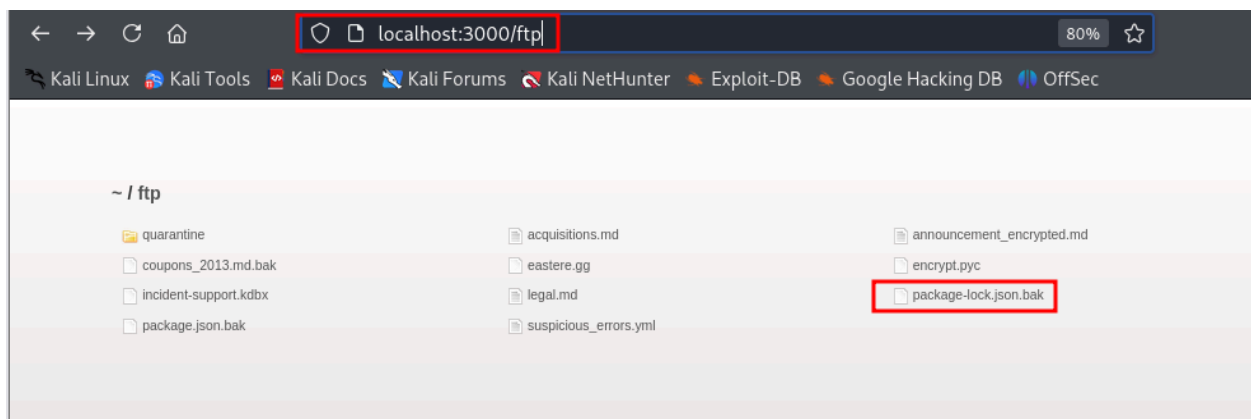
17. Null Byte Injection Allows Unauthorized Download of Backup Files

Affect on

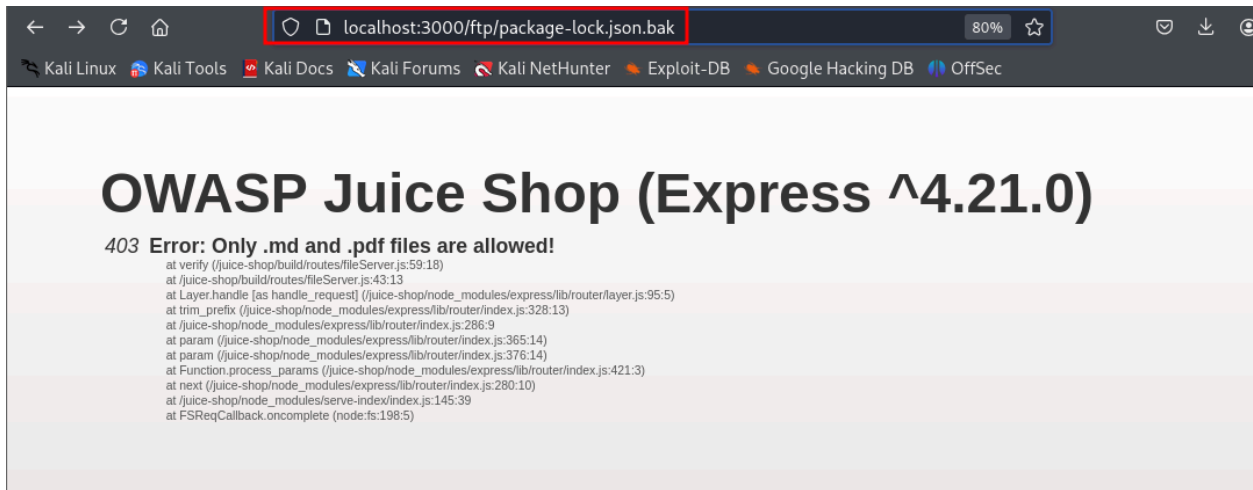
Infected URL	http://localhost:3000/ftp/package-lock.json.bak%2500.md
Infected Parameter	File path in the /ftp/ endpoint
Parameter type	Path parameter
Attack vector	Appending a URL-encoded null byte %2500 plus .md to the .bak filename bypasses extension checks, allowing download of restricted backup files.

- **Severity: Medium** — Exposure of sensitive backup files can assist attackers in mapping the application's structure and identifying vulnerable dependencies.
- **Ease of Exploitation: Easy** — Can be exploited using only a browser or simple HTTP request, no authentication required.

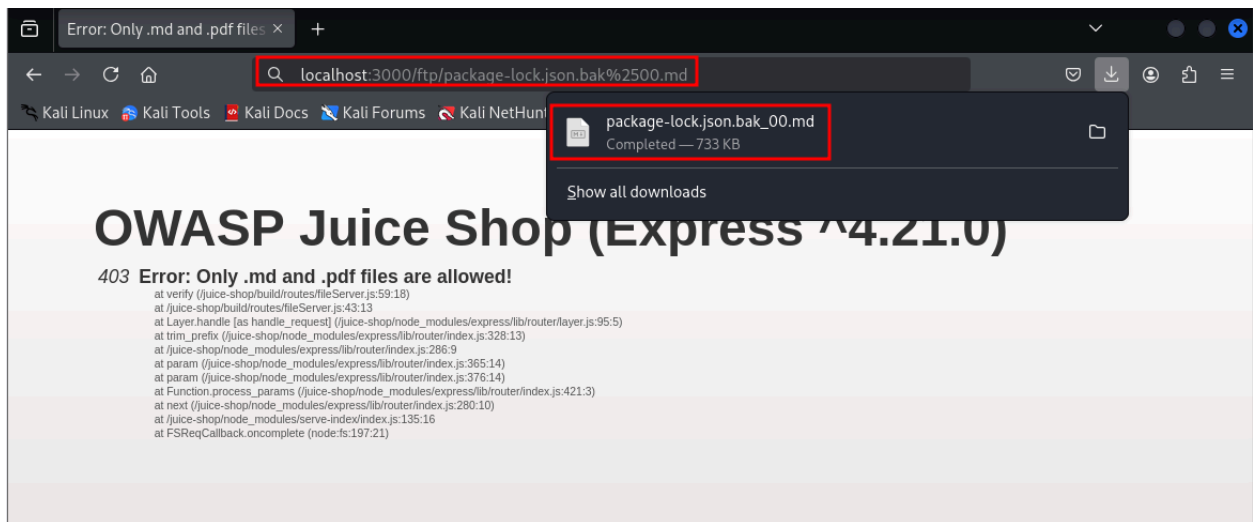
Analysis



While testing the /ftp/ endpoint, I tried to download the package-lock.json.bak file directly via the URL



However, the server returned an error and refused to serve the file, indicating that .bak files are restricted from download.



To bypass this restriction, I appended a URL-encoded null byte followed by .md to the filename:
`http://localhost:3000/ftp/package-lock.json.bak%2500.md`

Here, %2500 corresponds to the URL-encoded null byte (%00). This tricks the server's file extension validation logic, which only checks the suffix after the null byte (in this case .md, which is allowed).

As a result, the server served the .bak file despite the .md extension check. This indicates that the application's filename validation is vulnerable to null byte injection, allowing unauthorized access to backup files that should be restricted. The downloaded package-lock.json.bak file contained full dependency details, including exact versions and package sources.

Root Cause:

- Insecure filename validation logic that does not account for null byte characters.
- Relying solely on string checks without sanitizing or normalizing input.

Impact:

- Unauthorized access to sensitive backup files (.bak, .old, .tmp).
- Disclosure of package dependency versions from package-lock.json.bak.
- Potential identification of outdated libraries that could be exploited further.

Mitigation:

1. Implement strict server-side whitelisting of allowed file extensions.
2. Sanitize and normalize all input to remove null byte characters before processing.
3. Block access to backup or temporary files via server configuration (e.g., .bak, .old, .tmp).
4. Use framework-provided safe file handling functions rather than custom string checks.

18. Broken Access Control – Unauthorized Deletion of 5-Star Reviews

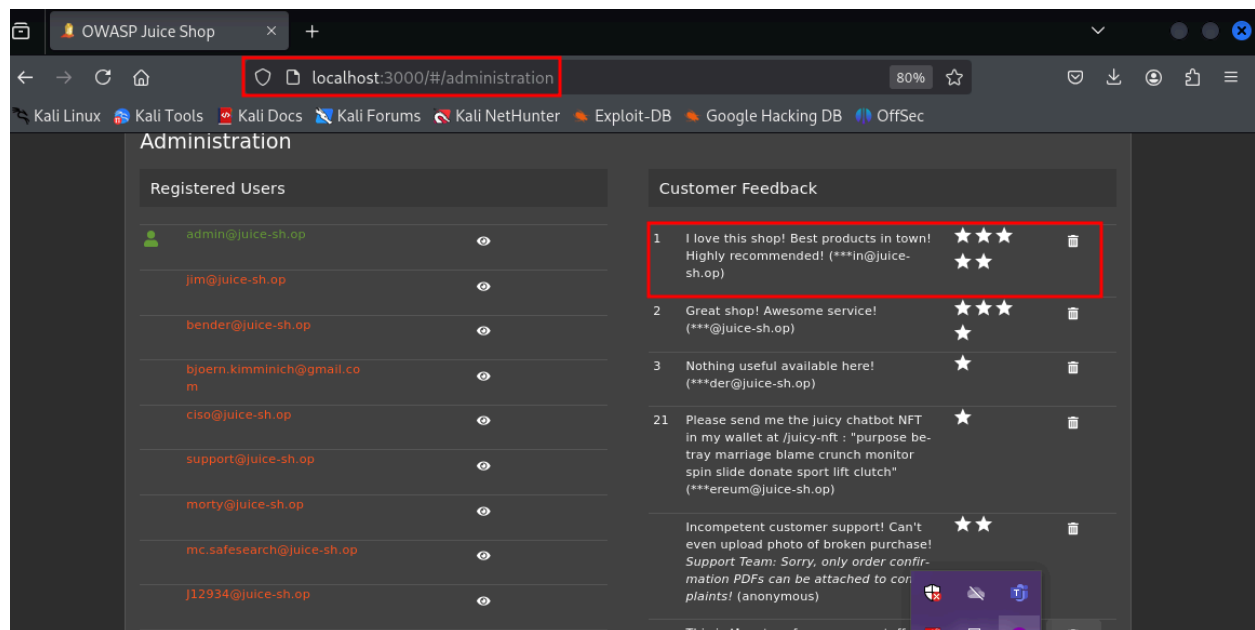
Affect on

Infected URL	http://localhost:3000/#/administration
Infected Parameter	No explicit parameter in URL, but actions performed through the admin interface trigger backend API calls to delete reviews.
Parameter type	Delete
Attack vector	Authenticated user deletes other users' 5-star reviews via the admin interface or API without proper authorization checks.

- Severity:- Medium
- Ease of Exploitation :- Easy

Analysis

During testing, I accessed the administration panel at <http://localhost:3000/#/administration> and was able to delete a 5-star review that did not belong to my user account. I found that the application does not properly enforce authorization checks on the review deletion functionality.



Root Cause

The application does not verify whether the user deleting the review is its owner or has administrative rights before processing the delete operation.

Impact

- Unauthorized deletion of product reviews.
- Potential to unfairly remove positive reviews or sabotage competitors.
- Data integrity and user trust compromised.

Mitigation

- Implement strict authorization checks on the DELETE /api/Reviews/:reviewId endpoint.
- Verify that the authenticated user is the owner of the review or has sufficient privileges before allowing deletion.
- Implement role-based access control (RBAC) to restrict actions based on user roles.
- Log all deletion actions for auditing purposes.

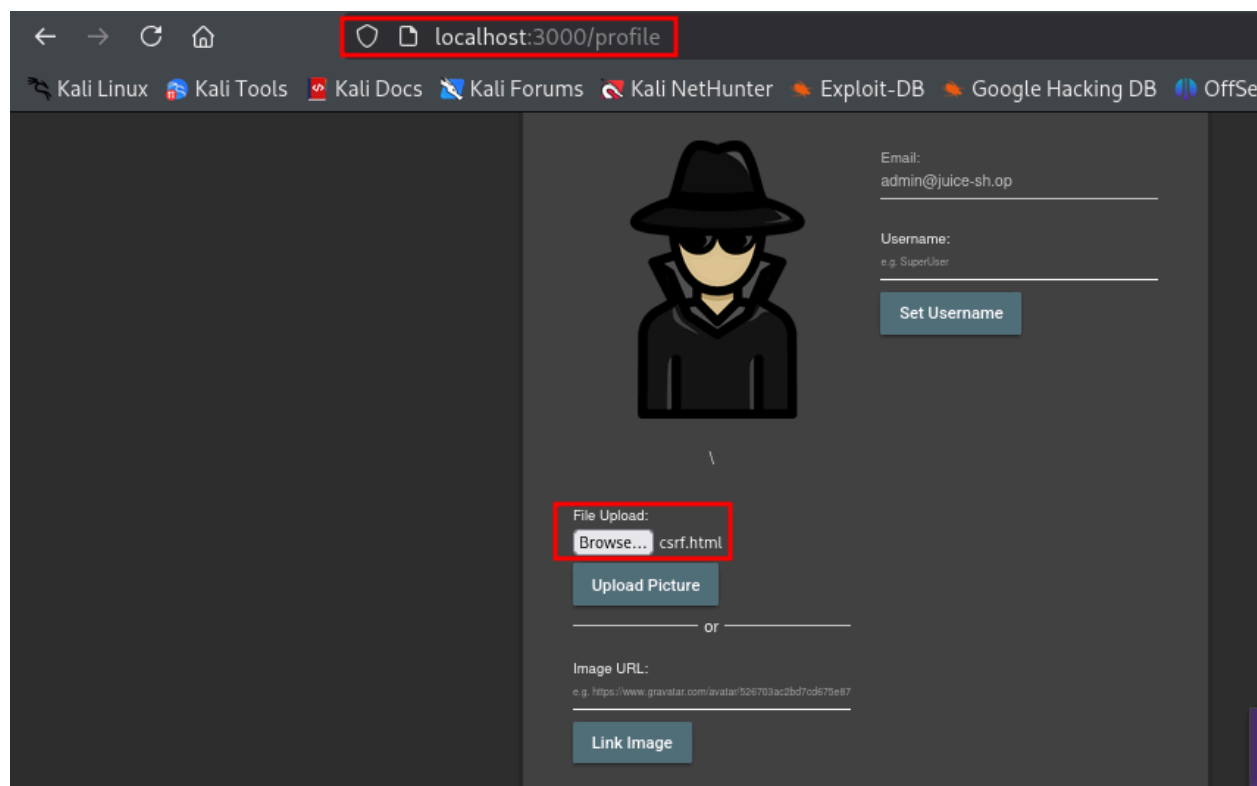
19. Verbose Error Message Revealing Internal Implementation Details

Affect on

Infected URL	http://localhost:3000/profile/image/file
Infected Parameter	file (uploaded file parameter)
Parameter type	File upload (multipart/form-data)
Attack vector	A file with an unsupported or disallowed file type to the profile image upload endpoint. The server responds with a 500 Internal Server Error containing a stack trace and sensitive internal details.

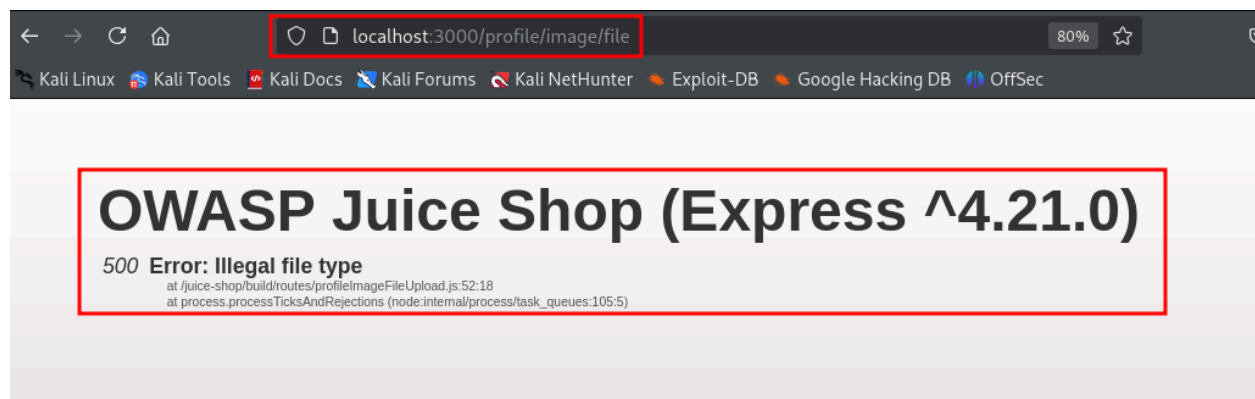
- **Severity: Medium** (could be escalated to High if disclosed version has known exploits)
- **Ease of Exploitation: Easy** — only requires submitting a crafted file upload request.

Analysis



The application's file upload endpoint does not handle exceptions securely. When an invalid file type is uploaded, the error is returned **verbatim** to the client, including:

- Full file path (/juice-shop/build/routes/profileImageFileUpload.js)
- Line number where error occurred (:52:18)
- Framework version (Express ^4.21.0)
- Node.js internal stack trace



These details provide attackers with useful reconnaissance for further exploitation

Root Cause:

The backend does not sanitize error output before sending it to the client. Instead of returning a generic error message, it leaks internal implementation details through unhandled exceptions or verbose logging.

Impact:

- Disclosure of **framework and library versions** → can be matched to known CVEs.
- Disclosure of **file system structure** → aids path traversal or LFI attacks.
- Potential for **targeted exploitation** if vulnerable software is identified.

Mitigation:

1. **Generic Error Messages:** Configure Express to send only generic responses like *"Invalid file type"* to the client.
2. **Error Logging:** Log full error details **server-side only** for debugging, not to the client.
3. **Secure Headers:** Ensure headers like X-Powered-By are removed to reduce tech stack disclosure.
4. **Exception Handling Middleware:** Implement global error handling in Express to catch and sanitize all errors.

20. Unauthenticated Access to Application Metrics Endpoint

Affect on

Infected URL	http://localhost:3000/metrics
Infected Parameter	No input parameter — direct endpoint access.
Parameter type	Endpoint path (HTTP GET request).
Attack vector	An attacker can directly access the /metrics endpoint without authentication.

- Severity:- Medium

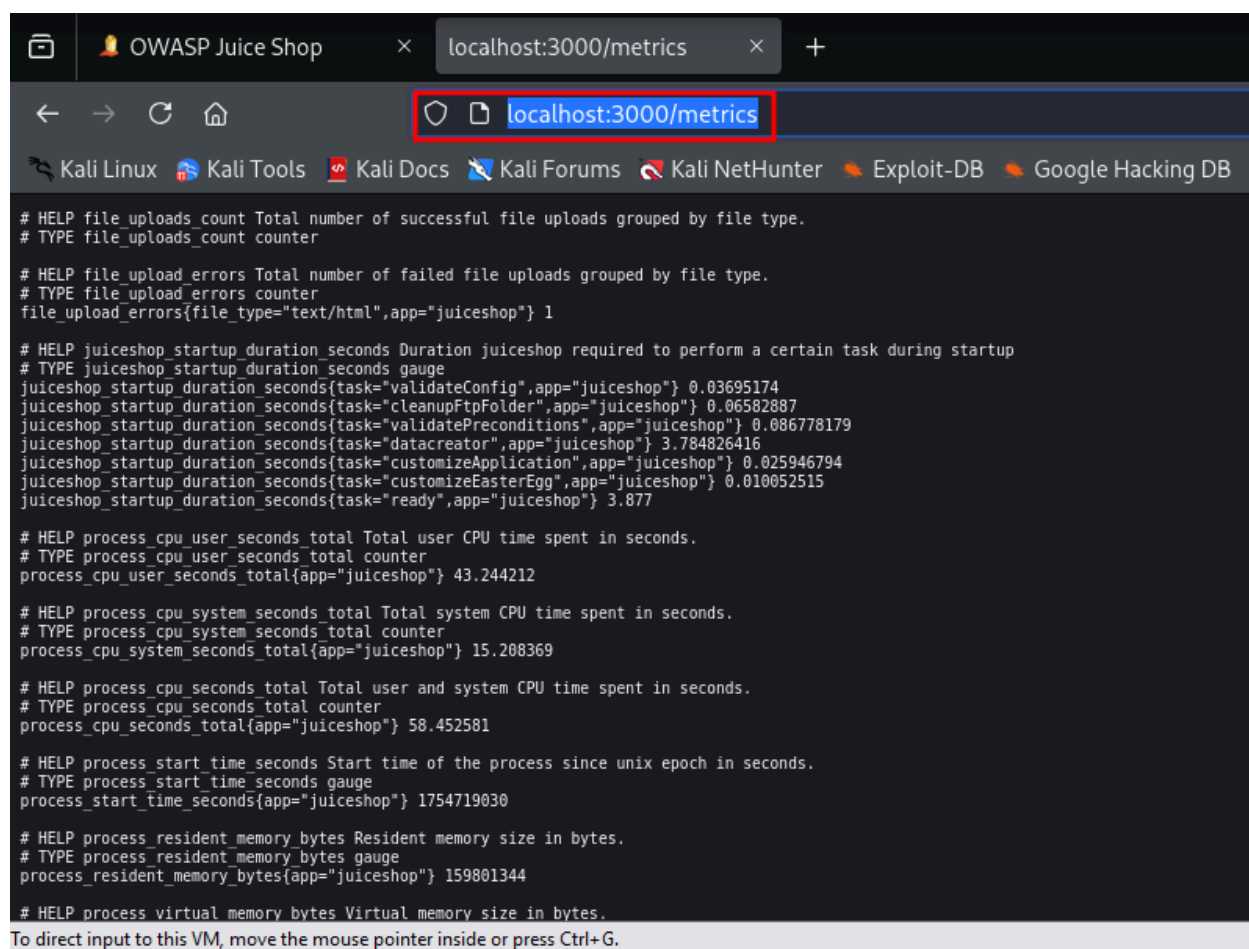
While this does not directly compromise user accounts, it provides valuable reconnaissance data that can facilitate further attacks such as DoS, parameter fuzzing, or targeted exploitation.

- Ease of Exploitation :- Easy

No authentication or special tools are required; the attacker simply visits the endpoint in a browser or via curl.

Analysis

The /metrics endpoint is designed for internal monitoring tools like Prometheus. In Juice Shop, it is exposed publicly without any authentication or network restriction. This allows anyone on the internet (or local network) to view sensitive operational data.



```
# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter
file_upload_errors{file_type="text/html",app="juiceshop"} 1

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 0.03695174
juiceshop_startup_duration_seconds{task="cleanupFtpFolder",app="juiceshop"} 0.06582887
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 0.086778179
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 3.784826416
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.025946794
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.010052515
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 3.877

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 43.244212

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total{app="juiceshop"} 15.208369

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total{app="juiceshop"} 58.452581

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds{app="juiceshop"} 1754719030

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes{app="juiceshop"} 159801344

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Root Cause

The metrics endpoint is not restricted to authenticated users or internal networks. There is no access control mechanism in place, and sensitive operational data is served to any request.

Impact

- Exposure of application internals such as uptime, memory usage, and request patterns.
- Potential leakage of sensitive metric labels if developers included user-specific or business data.
- Attackers can use performance stats to:
 - Time DoS attacks more effectively.
 - Identify endpoints with high error rates to target for exploitation.

Mitigation

1. Restrict access to /metrics to internal IP ranges or require admin authentication.
2. Remove sensitive data from custom metrics.
3. Disable metrics endpoint in production unless strictly necessary.
4. Use HTTPS to ensure metrics are not leaked over insecure channels.

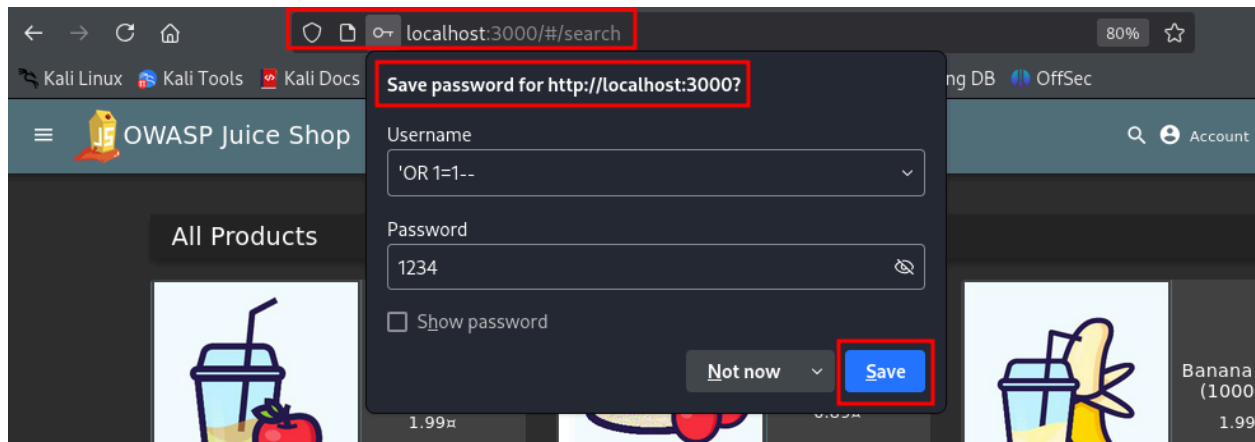
21. Insecure Autocomplete Configuration on Login Form

Affect on

Infected URL	http://localhost:3000/#/search
Infected Parameter	N/A (Browser behavior tied to form fields)
Parameter type	Form input fields – email and password
Attack vector	There is no autocomplete="off" attribute set on the form or its input fields.

- Severity: Low
- **Ease of Exploitation: Trivial** – This behavior occurs automatically in most modern browsers when autocomplete is not disabled.

During testing, after logging into the Juice Shop application, the browser displayed a prompt to save the entered credentials. This behavior indicates that the application does not restrict autocomplete on the login form fields.



In environments where users access the application from **shared or public computers**, this may lead to unintended exposure of credentials to unauthorized users.

Root Cause:

Lack of appropriate use of autocomplete="off" or autocomplete="new-password" attributes on sensitive input fields.

Impact:

- In shared or public environments, saved credentials may be auto-filled for the next user, leading to unauthorized access.
- Increases risk of phishing or social engineering attacks if credentials are exposed.
- May result in sensitive information leakage if browser-saved credentials are extracted via malware or local attacks.

Mitigation:

To reduce this risk, the application should disable autocomplete functionality on all sensitive form fields, particularly those related to authentication and personal data.

Recommended Fix:

Update the login form as follows:

```
<form autocomplete="off">  
  <input type="text" name="email" autocomplete="off">  
  <input type="password" name="password" autocomplete="new-password">  
</form>
```

- Use autocomplete="off" on the form and username field
- Use autocomplete="new-password" on the password field

Conclusion

The security assessment of the OWASP Juice Shop application uncovered multiple vulnerabilities that reflect common real-world security risks, as highlighted by the OWASP Top 10 framework. These vulnerabilities demonstrate how attackers can exploit issues such as injection flaws, broken access control, and insecure configurations to compromise the system's integrity and confidentiality.

Through this testing exercise, critical weaknesses were identified, providing valuable insights into the importance of secure coding practices, proper input validation, and robust access controls. Addressing these vulnerabilities by implementing the recommended mitigations will significantly improve the security posture of the application.

Furthermore, this exercise emphasizes the need for continuous security assessments and developer awareness to prevent similar issues in real-world applications. The OWASP Juice Shop serves as an effective training ground for understanding, detecting, and remediating common vulnerabilities, ultimately fostering a culture of security-first development.